# Kotlin Training Setup Instructions

- Follow the following setup instructions prior to the training
- The IDE used for the course should be Intellij.
- If there are any questions please contact me:
  Urs Peter / upeter@xebia.com

  I'm glad to help.

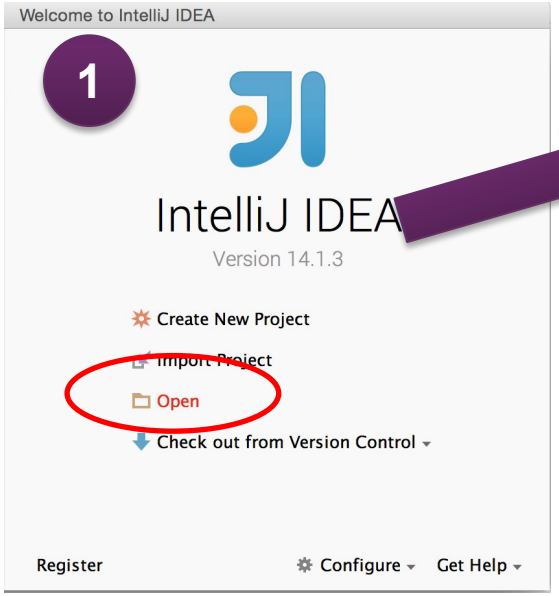
# Setup Instructions for Intellij

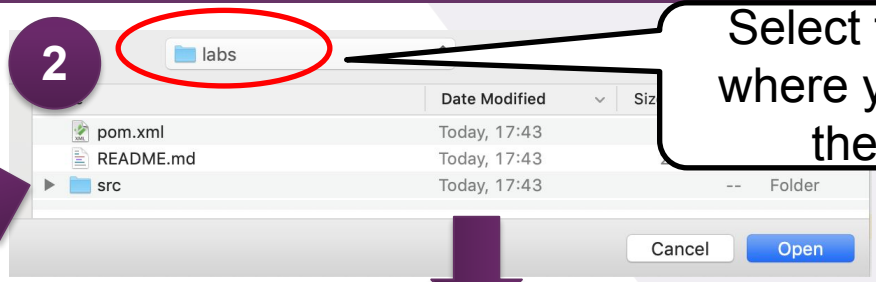
- Make sure you have JDK >=21 installed
- Make sure you have the latest version of Intellij installed

  If not: download Intellij here - the community edition is sufficient:
  - https://www.jetbrains.com/idea/download/#section=windows
- Import lab sources *(also see screenshots)*
  - Download and unzip:
    - https://bit.ly/kotlinconf-springboot-workshop-labs-2026
  - In the lab root directory run:

    ```
    ./mvnw clean test
    ```
    - All tests will fail
  - Import Project in Idea:
    - File -> Open-> select lab root directory -> Ok
    - Install the Kotest plugin found under:
      - Preferences -> Plugins -> Marketplace
    - From within the IDE make sure you can run all kotlin and java tests

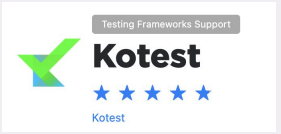
# Screenshots Project Import in Intellij



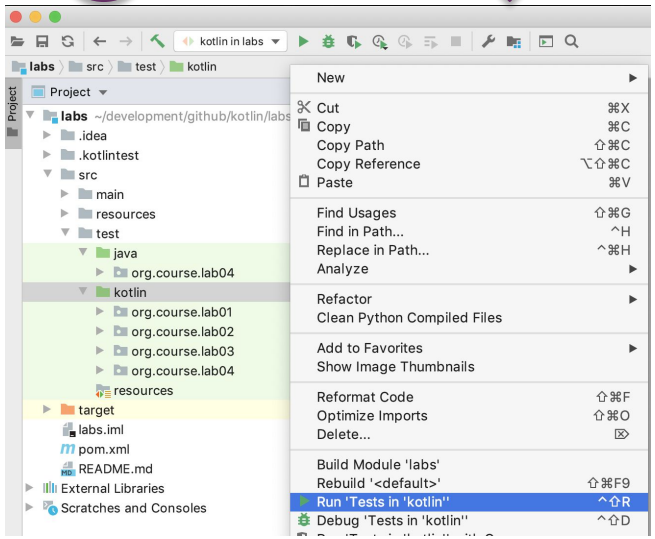




Select the directory where you unzipped the labs file

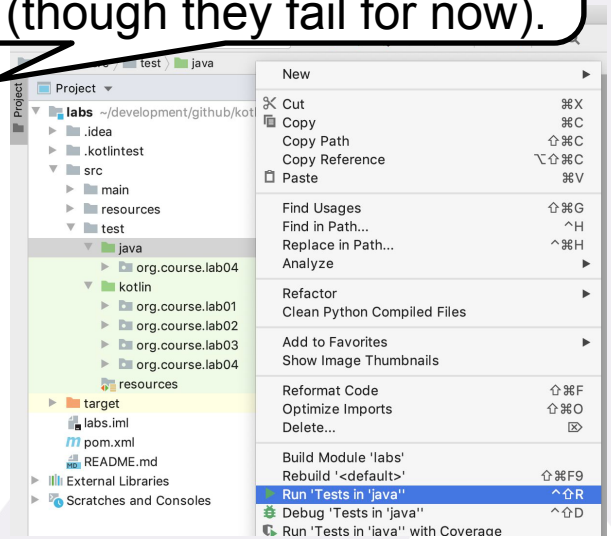
3 Install the Kotest plugin found under:
*Preferences -> Plugins -> Marketplace*




4




Make sure you can run the java and kotlin tests (though they fail for now).



# KotlinConf 2026 Workshop

## Reactive Spring Boot with Coroutines

## &

## Virtual Threads




Urs Peter
upeter@xebia.com

# About Me:


Kotlin Training
Certified by JetBrains

Xebia Academy



# Nice to meet:

# Sources and Slides

- Slides:
  - https://bit.ly/kotlinconf-springboot-workshop-slides-2026
- Sources:
  - https://bit.ly/kotlinconf-springboot-workshop-labs-2026

# Agenda Advanced Kotlin

- Coroutines Basics: Structured Concurrency
- Coroutines Basics: Contexts + Propagation

- Coroutines & Spring Boot: Introduction
- Coroutines & Spring Boot: Reactive DB Access & REST calls with Coroutines
- Coroutines & Spring Boot: Reactive Controllers with Coroutines

- Coroutines: Flow
- Coroutines & Spring Boot: Reactive Stream Controllers with Flow (SSE and Websockets)

- Virtual Threads &| Coroutines

# Agenda

- Coroutines Basics: Structured Concurrency
- Coroutines Basics: Contexts + Propagation

- Coroutines & Spring Boot: Introduction
- Coroutines & Spring Boot: Reactive DB Access & REST calls with Coroutines
- Coroutines & Spring Boot: Reactive Controllers with Coroutines

- Coroutines: Flow
- Coroutines & Spring Boot: Reactive Stream Controllers with Flow

- Virtual Threads &| Coroutines

# Sequential Programming...

```java
public URL randomAvatar() { ... }          //remote blocking method call
public Boolean verifyEmail(String name) { ... } //remote blocking method call
public User save(User user) {... }         //remote blocking db method call
```

Java

```java
@PostMapping("/users")
@ResponseBody
public User storeUser(@RequestBody User user) {
    var avatarUrl  = randomAvatar()
    var validEmail = verifyEmail(user.getEmail());
    if(!validEmail) {
        throw new InvalidEmailException("Invalid Email");
    }
    return save(UserBuilder.from(user).withAvatarUrl(avatarUrl).build());
}
```

Java

# Async on the other hand…

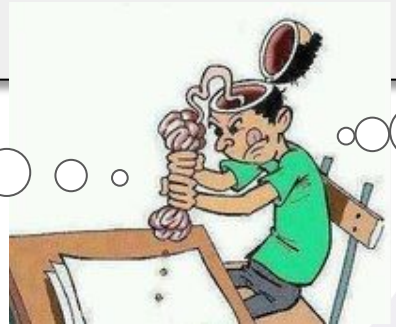
CompletableFutures, RxJava, Reactor etc. offer *building blocks* for *async programming*.

```
public Mono<Boolean> verifyEmail(String name) { ... } //remote async method call
public Mono<URL> randomAvatar() { ... } //remote async method call
public Mono<User> save(User user)
```

Since the async building blocks are *libraries* - and not *language features* - they force you to tightly bind your code to their API and abstractions.

```
@PostMapping("/users")
@ResponseBody
public Mono<User> storeUser(@RequestBody User user) {
 Mono<URL> avatarMono = avatarService.randomAvatar();
 Mono<Boolean> validEmailMono = emailService.verifyEmail(user.getEmail());
  return Mono.zip(avatarMono, validEmailMono).flatMap(tuple ->
   if(!tuple.getT2()) //what is getT2()? It's the validEmail Boolean...
       Mono.error(new InvalidEmailException("Invalid Email"));
   else personRepo.save(UserBuilder.from(user)
                                   .withAvatarUrl(tuple.getT1()));
  );
}
```

The ***business intent*** of my code gets **lost** in all the 'combinator jungle' ☹



...and this stuff is *quite hard* **to learn** and **handle correctly** too ☹

# Kotlin Coroutines to the rescue!

The concurrency building blocks of Kotlin rely on *Coroutines*.

With Coroutines, logic can be expressed *sequentially* whereas the underlying implementation figures out the *asynchrony*.

A method marked `suspend` can be run *within* a *coroutine* that can suspend it without blocking a Thread

```
suspend fun randomAvatar(): URL = ...
suspend fun verifyEmail(email:String): Boolean = ...
suspend fun save(user:User): Long = …
```

```
@GetMapping("/users")
@ResponseBody
suspend fun storeUser(@RequestBody user:User): User {
    val avatarUrl  = randomAvatar()
    val validEmail = verifyEmail() }
    if(!validEmail) throw InvalidEmailException("Invalid email")
    return save(user.copy(avatar = avatarUrl))
}
```

*Wow, even my boss understands this code!*

# Kotlin Coroutines to the rescue!

The concurrency building blocks of Kotlin rely on *Coroutines*.

With Coroutines, logic can be expressed *sequentially* whereas the underlying implementation figures out the *asynchrony*.

???

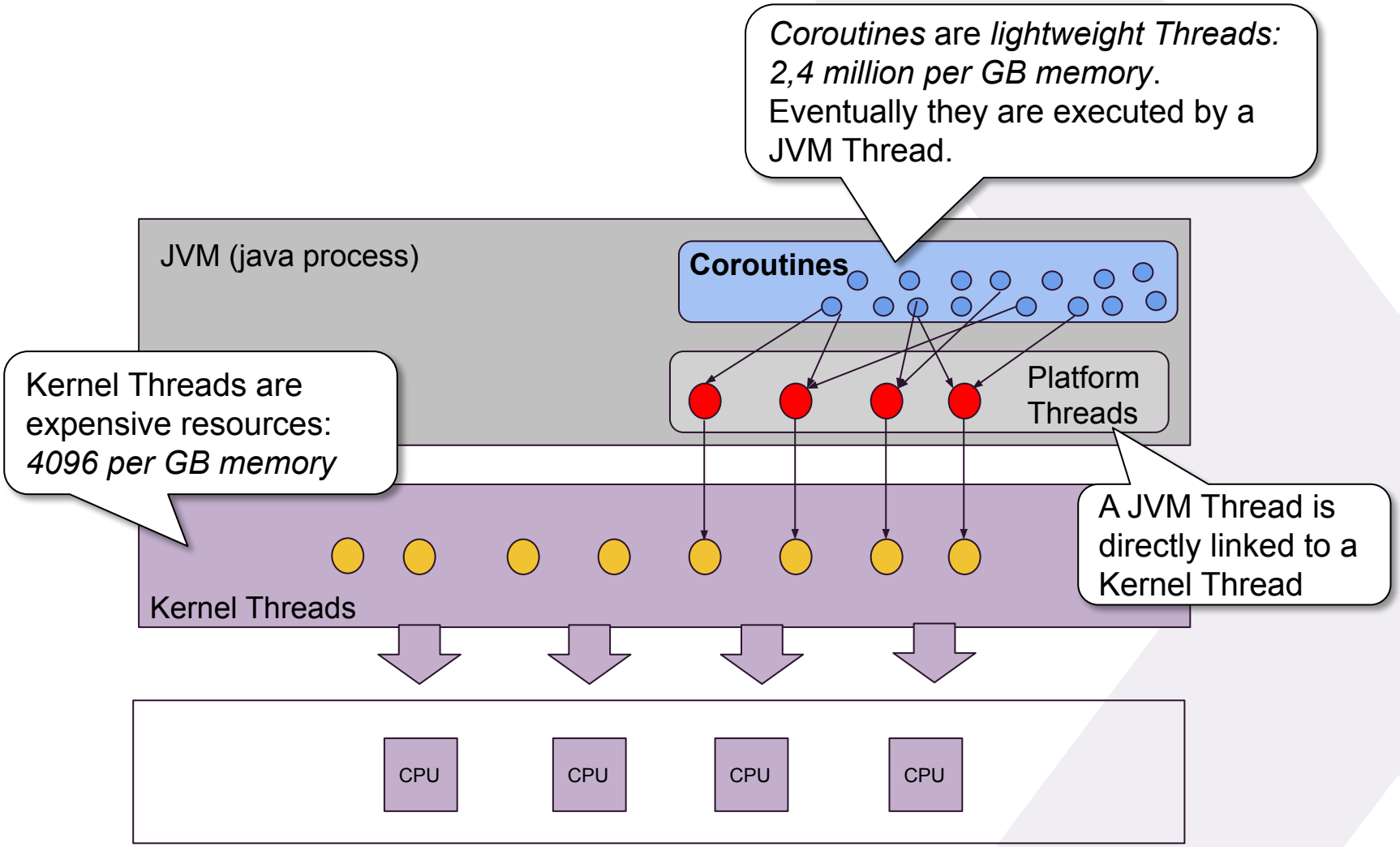
```
suspend fun randomAvatar(): URL = ...
suspend fun verifyEmail(email:String): Boolean = ...
suspend fun save(user:User): Long = …
```

???

```
@GetMapping("/users")
@ResponseBody
suspend fun storeUser(@RequestBody user:User): User = coroutineScope {
    val avatarUrl  = async{ randomAvatar() }
    val validEmail = async{ verifyEmail() }
    if(!validEmail.await()) throw InvalidEmailException("Invalid email")
    return save(user.copy(avatar = avatarUrl.await()))
}
```

*Even better - now it processes in **parallel**!*

# What are Coroutines?



# From Threads to Coroutines

We create a Thread using Kotlin's helper method

```
val thread = thread {
    sleep(500)
    println("Rabbit")
}
println("Turtle")
thread.join()
```

...and use `join()` to wait for the result by blocking the current thread

`runBlocking` is a special builder that blocks the calling Thread until all Coroutines have been terminated

`launch` creates a Coroutine

`delay()` will *suspend* the Coroutine

```
runBlocking {
    launch {
        delay(500)
        println("Rabbit")
    }
    println("Turtle")
}
```

Coroutines are created by means of Coroutine Builder methods (here `runBlocking` and `launch`). ***Important**: Avoid using `runBlocking` in production code, limit its use for testing purposes*

# So what's the difference?

```
val threads = (1..1_000_000).toList().map {
   thread {
      if(it % 1000 == 0) println("Created $it threads")
      sleep(10000)
      print(".")
   }
}
threads.forEach{it.join()}
```


```
   runBlocking {
       (1..1_000_000).toList().map {
         launch {
            if(it % 1000 == 0) println("Created $it coroutines")
            delay(10000)
            print(".")
         }
       }
   }
```

# Suspending Functions

The heart of coroutines are *suspend functions* marked with the `suspend` keyword

They look like 'normal functions' but can be *paused* and *resumed at a later time* by a Coroutine *without blocking*.

```
suspend fun delay(timeMillis: Long): Unit = ...
```

Suspending functions can only be called by another *suspending function* or a *Coroutine*

```
fun main() {
  delay(1000)
}
```

Compilation Error

```
launch {
    delay(1000)
}
```

```
suspend fun easyOn() {
    delay(1000)
}
```

# Coroutine Builders

To create Coroutines Kotlin offers a variety of Coroutine builders:

```
fun <T> runBlocking(...): T

fun CoroutineScope.launch(...):Job

fun <T> CoroutineScope.async(...):Deferred<T>
```

# Coroutines Core Ingredients: CoroutineScope & CoroutineContext

All Coroutine builder method provide two mandatory objects needed for a Coroutine to run: `CoroutineContext` and `CoroutineScope`.

Various builder methods

```
fun <T> runBlocking|launch|async(
        context: CoroutineContext = EmptyCoroutineContext,
        block: suspend CoroutineScope.() -> T
): T
```

```
val x = runBlocking {
    println("Wait a second...")
    delay(1000)
    println("This is: $this")
}
println("value is $x")
```

For now: what is the type of `this`?

# Coroutine Builders: launch

`CoroutineScope` has an extension method `launch`

`launch` creates a Coroutine that only produces a side-effect (no return value) from the executed code.

```
fun CoroutineScope.launch(
        context: CoroutineContext = EmptyCoroutineContext, ...
        block: suspend CoroutineScope.() -> Unit): Job
```

# launch: inherited vs new CoroutineScope

- A CoroutineScope controls the lifecycle of a Coroutine.
- Scopes can be nested.
- Each *parent* scope coordinates the completion of the Coroutine(s) in its *child* scope.
- This is called ***structured concurrency*** and basically omits the need for manual synchronization like in Threads, which is great!

`launch` inherits its `CoroutineScope` from the parent (`runBlocking`). This makes sure that `runBlocking` automatically waits for the completion of the Coroutine executed by `launch`

Parent scope

Child scope

```
runBlocking { this:CoroutineScope
  this.launch { this:CoroutineScope
    delay(500)
    println("Rabbit")
  }
  println("Turtle")
}
```

The `GlobalScope` is a `CoroutineScope` `object`. `launch` starts a *new* root `CoroutineScope` and does not inherit it from `runBlocking`. Therefore, no synchronization takes place.

Parent scope

*new* Parent scope

```
runBlocking { this:CoroutineScope
  GlobalScope.launch { this:CoroutineScope
    delay(500)
    println("Rabbit")
  }
  println("Turtle")
}
```

# launch: fine-grained Structured Concurrency with CoroutineScope

- `coroutineScope{...}` is a `suspend` method that provides a new `CoroutineScope`, which will automatically ensure that all children complete before it proceeds.
- `coroutineScope{...}` is a 'normal' `suspend` method like `delay(...)`.
- `suspend` methods are *always executed in sequential order* (yet non-blocking).
- This building block allows *parallel decomposition* of work.

How can we print Rabbit *before* Turtle, even though printing Rabbit is delayed in a separate Coroutine?

Parent scope

Child scope

```
runBlocking { this:CoroutineScope
  this.launch { this:CoroutineScope
    delay(500)
    println("Rabbit")
  }
  println("Turtle")
}
```

…simply wrap the 'process tree' you want to be terminated before continuing in a `coroutineScope{...}` block

Parent scope

Child scope

```
runBlocking {
  coroutineScope { this:CoroutineScope
    this.launch { this:CoroutineScope
      delay(500)
      println("Rabbit")
    }
  }
  println("Turtle")
}
```

# Coroutine Builders: return type of launch

`launch` returns a handle to the Coroutine in the form of a `Job` instance.

```
fun CoroutineScope.launch(...):Job
```

The `Job` knows the state of the Coroutine. Besides that it allows Coroutines to be canceled or joined.

```
class Job {
    val isActive: Boolean
    val isCompleted: Boolean
    val isCancelled: Boolean
    fun start(): Boolean
    fun cancel()
    suspend fun join()
}
```

```
runBlocking { this:CoroutineScope
    val job = this.launch {this:CoroutineScope
        delay(500)
        println("Rabbit")
    }
    println("Arrived? ${job.isCompleted}")
}
```

What is the value of *isCompleted*?

# Coroutine Builders: return type of launch

```
fun GlobalScope.launch(...):Job
```

```
class Job {
    val isActive: Boolean
    val isCompleted: Boolean
    val isCancelled: Boolean
    fun start(): Boolean
    fun cancel()
    suspend fun join()
}
```

```
runBlocking { this:CoroutineScope
    val job = GlobalScope.launch {
        delay(500)
        println("Rabbit")
    }
    println("Arrived? ${job.isCompleted}")
    job.join()
    println("Arrived! ${job.isCompleted}")
}
```

Since *GlobalScope* is not bound to its parent scope, manual synchronization is needed by using *job.join()*

# Coroutine Builders: async

`async` executes a Coroutine that produces a return value

```
fun <T> CoroutineScope.async(
            context: CoroutineContext = EmptyCoroutineContext, ...
            block: suspend CoroutineScope.() -> T
):Deferred<T>
```

The return value is wrapped in non-blocking cancelable future called `Deferred<T>` that extends from `Job`.

```
class Deferred<out T>:Job {
    suspend fun await():T
}
```

```
runBlocking { this:CoroutineScope
    val rabbit = this.async {this:CoroutineScope
        delay(500)
        "Rabbit".also(::println)
    }
    println(
      "Turtle meets: ${rabbit.await()}"
    )
}
```

`await()` waits until the result is available without blocking.

# Coroutine Builders: async

`async` executes a Coroutine that produces a return value

```
fun <T> CoroutineScope.async(
            context: CoroutineContext = EmptyCoroutineContext, ...
            block: suspend CoroutineScope.() -> T
):Deferred<T>
```

```
suspend fun meet() = coroutineScope { this:CoroutineScope

    val rabbit = this.async {this:CoroutineScope
        delay(500)
        "Rabbit".also(::println)
    }
    println(
      "Turtle meets: ${rabbit.await()}"
    )
}
```

`async{...}` and `launch{...}` require a `CoroutineScope`.
The `coroutineScope {...}` block can be used to get a scope that is deduced from the `suspend` method.

# suspend vs launch / async

What is the difference between the two snippets?

```
suspend fun meet() = coroutineScope {
    val rabbit = async {
        delay(500)
        "Rabbit"
    }
    println(
      "Turtle meets: ${rabbit.await()}"
    )
}
```

```
suspend fun meet() {
    delay(500)
    val rabbit = "Rabbit"
    println("Turtle meets: $rabbit")
}
```

Bottom line:

- only use `async` / `launch` if you want to do something in *parallel*
  - this means *at least* two *operations* at the *same time*

- If you only want *non-blocking behavior* a `suspend` method is enough

# Exercise: Coroutines


- Slides:
  - https://bit.ly/kotlinconf-springboot-workshop-slides-2026
- Sources:
  - https://bit.ly/kotlinconf-springboot-workshop-labs-2026

- Open: Coroutines01BasicsExercise.kt and Coroutines01BasicsExerciseTest.kt
- **Solve Exercise: A-D**

  - `Coroutines01BasicsExerciseTest.kt`: here you find the instructions of all exercises, which most have to be completed right away in the test.

  - `Coroutines01BasicsExercise.kt`: in this file you have to provide an implementation, which will be used in all exercises.
    - **Important**: Ensure that in the `getCurrency(...)` method you literally log the statement: `logger.info("Get currency ~~blocking~~ $currency at rate $rate")` as indicated in the instructions. Otherwise, tests will fail.

You have completed the challenge if all tests succeed in `Coroutines01BasicsExerciseTest`.

# Agenda Advanced Kotlin

- Coroutines Basics: Structured Concurrency
- Coroutines Basics: Contexts + Propagation

- Coroutines & Spring Boot: Introduction
- Coroutines & Spring Boot: Reactive DB Access & REST calls with Coroutines
- Coroutines & Spring Boot: Reactive Controllers with Coroutines

- Coroutines: Flow
- Coroutines & Spring Boot: Reactive Stream Controllers with Flow

- Virtual Threads &| Coroutines

# Coroutine Builders Core Ingredients: CoroutineContext

Various builder methods. As said: use `runBlocking` only for testing purposes

`CoroutineContext` offers the Context needed to execute the Coroutine like: `Dispatchers` (ThreadPool), name, `ExceptionHandler` etc.

```
fun <T> runBlocking|launch|async(
        context: CoroutineContext = EmptyCoroutineContext,
        block: suspend CoroutineScope.() -> T
): T
```

By default, a *child* Coroutine inherits the `CoroutineContext` from its *parent*.

# CoroutineContext

The only property of the `CoroutineScope` is a `CoroutineContext`

Under the hood the `CoroutineContext` is a `Map` containing various elements that provide the context needed to execute the Coroutine.

```
interface CoroutineScope {
    val coroutineContext: CoroutineContext

    fun plus(context: CoroutineContext): CoroutineScope =
                ContextScope(coroutineContext + context)
}
```

A new child `CoroutineScope` is created by merging a current context (parent) with a child context.

Key elements of the `CoroutineContext` are:

- `CoroutineDispatcher`
  - determines the thread on which a coroutine runs
  - as a `ContinuationInterceptor` intercepts coroutine continuations to control which thread the coroutine resumes on.
- `Job` handle to the Coroutine
- `CoroutineExceptionHandler`
- `CoroutineName`

Every element is a `CoroutineContext` by itself

# CoroutineContext

```
interface CoroutineScope {
    val coroutineContext: CoroutineContext

    fun plus(context: CoroutineContext): CoroutineScope =
                ContextScope(coroutineContext + context)
}
```

```
launch|async(CoroutineName("c1") + Dispatchers.IO + ...) { this:CoroutineScope
  println("${coroutineContext[CoroutineName.Key]} is on: ${Thread.currentThread().name}")
}
```

A particular `CoroutineContext` can be retrieved with its corresponding key.

```
suspend fun meet() = withContext(CoroutineName("c1") + Dispatchers.IO + ...) {...}
```

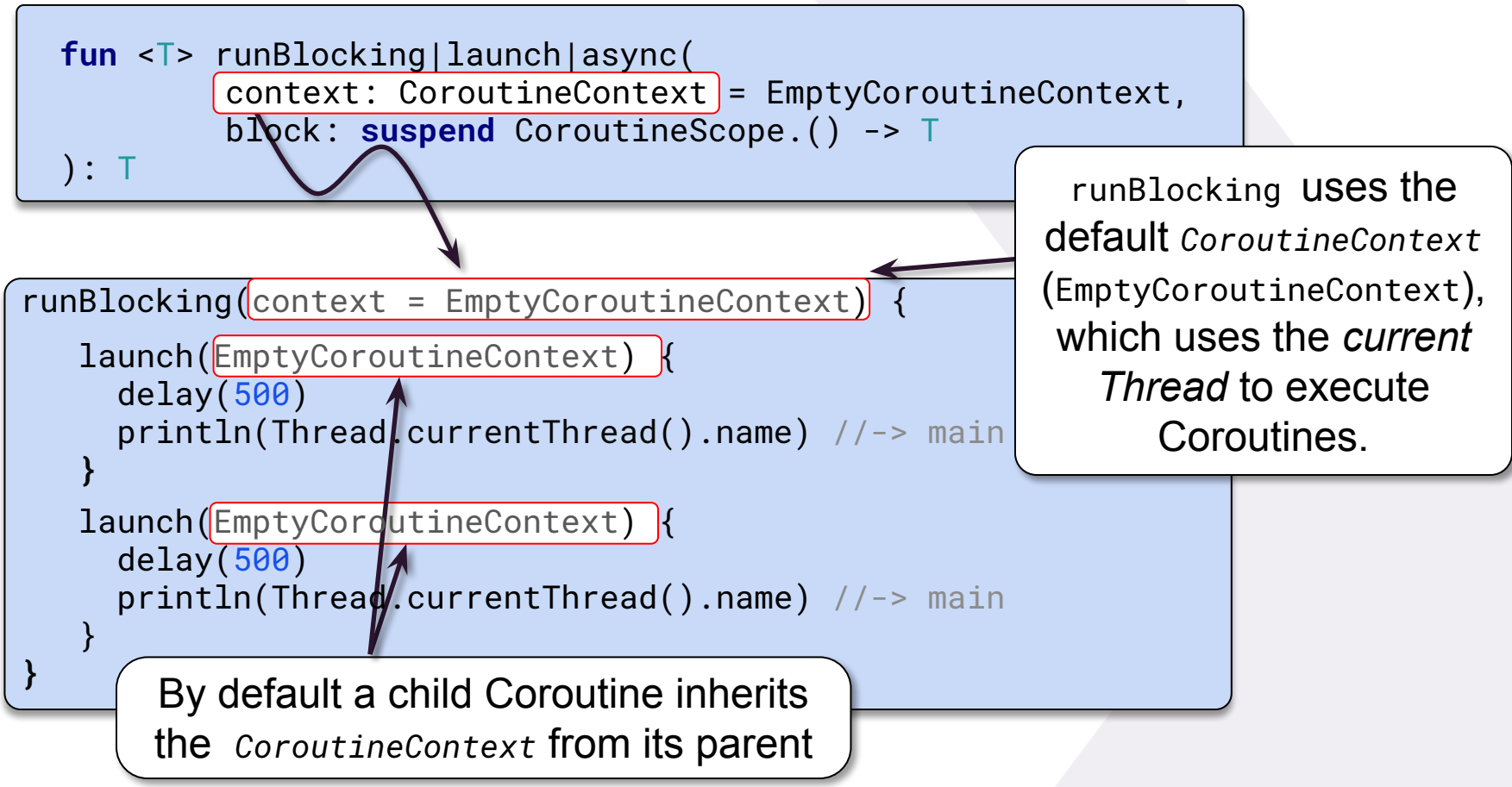
Within a `suspend` method, `withContext(...)` can be used to change the inherited `CoroutineContext`.

# CoroutineContext: Coroutine Dispatchers

Kotlin offers the following `CoroutineDispatchers` which are as we have seen `CoroutineContexts` themselves:

- `EmptyCoroutineContext` *(default)*
  - Uses the *current Thread* to execute all subsequent Coroutines.
- `Dispatchers.Default`
  - Good for CPU intensive tasks, backed by a shared pool of threads and limited by the number of CPU cores
- `Dispatchers.IO`
  - Used for blocking IO operations that are not CPU intensive
- `newSingleThreadContext("name")`
  - Single thread context
- `Dispatchers.Unconfined`
  - Executes coroutine in the current thread but continues after a suspension point in the thread that resumed it
- `Dispatchers.Main`
  - UI Thread Dispatcher mainly for Android

# Coroutine Dispatchers in Action



# Coroutine Dispatchers in Action

```
fun <T> runBlocking|launch|async(
        context: CoroutineContext = EmptyCoroutineContext,
        block: suspend CoroutineScope.() -> T
): T
```

```
runBlocking(context = EmptyCoroutineContext) {
    launch {
        sleep(500)
        println(Thread.currentThread().name) //-> main
    }
    launch {
        sleep(500)
        println(Thread.currentThread().name) //-> main
    }
}
```

**Tricky Question**: How long would it take to execute this code block using `Thread.sleep(...)` instead of `delay(...)`?

# Coroutine Dispatchers in Action

```
fun <T> runBlocking|launch|async(
        context: CoroutineContext = EmptyCoroutineContext,
        block: suspend CoroutineScope.() -> T
): T
```

```
runBlocking(context = Dispatchers.IO) {
    launch {
        sleep(500)
        println(Thread.currentThread().name) //-> worker-1
    }
    launch {
        sleep(500)
        println(Thread.currentThread().name) //-> worker-5
    }
}
```

…and how long would it take to execute this code block using the `Dispatchers.IO` (which contains 64 Threads)?

# ThreadLocal state propagation and Coroutines

```
fun <T> runBlocking|launch|async(
        context: CoroutineContext = EmptyCoroutineContext,
        block: suspend CoroutineScope.() -> T
): T
```

How about `ThreadLocal` state?
Will it be propagated too? 🤔

```
runBlocking(context = Dispatchers.IO) {
    MDC.put("id", "123")
    logger.info("Start Jobs")
    launch {
        sleep(500)
        logger.info("Job 1")
    }
    launch {
        sleep(500)
        logger.info("Job 2")
    }
}
```

[thread-**1**] INFO id=**123** - Start Jobs

[thread-**3**] INFO id=😐 - Job 1

[thread-**2**] INFO id=😐 - Job 2

# ThreadLocal state propagation and Coroutines

```
fun <T> runBlocking|launch|async(
        context: CoroutineContext = EmptyCoroutineContext,
        block: suspend CoroutineScope.() -> T
): T
```

```
runBlocking(context = Dispatchers.IO) {
    MDC.put("id", "123")
    logger.info("Start Jobs")
    withContext(MDCContext()) {
        launch {
            sleep(500)
            logger.info("Job 1")
        }
        launch {
            sleep(500)
            logger.info("Job 2")
        }
    }
}
```

[thread-**1**] INFO id=**123** - Start Jobs

[thread-**3**] INFO id=**123** - Job 1

[thread-**2**] INFO id=**123** - Job 2

This can easily be fixed with a `ThreadContextElement` implementation (which extends from `CoroutineContext`) that propagates state from `ThreadLocal` to a Coroutine and its subscope(s)

# Exercise: Coroutines


- Open: Coroutines01CoroutineContextExerciseTest.kt
- **Solve Exercise: A-B**
  - `Coroutines01CoroutineContextExerciseTest.kt`: here you find the instructions of all exercises, which most have to be completed right away in the test.
  - `Coroutines01BasicsExercise.kt`: reuse the implementation of `CurrencyService` you created in the previous exercise in the aforementioned test.

You have completed the challenge if all tests succeed in `Coroutines01CoroutineContextExerciseTest`.

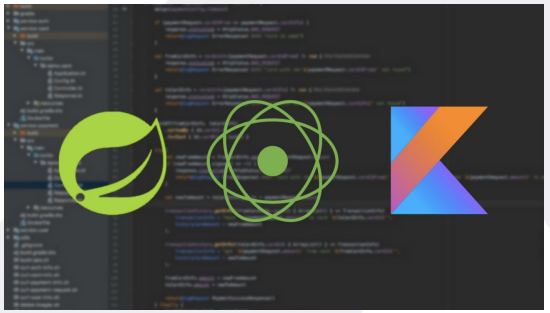
# Agenda

- Coroutines Basics: Structured Concurrency
- Coroutines Basics: Contexts + Propagation

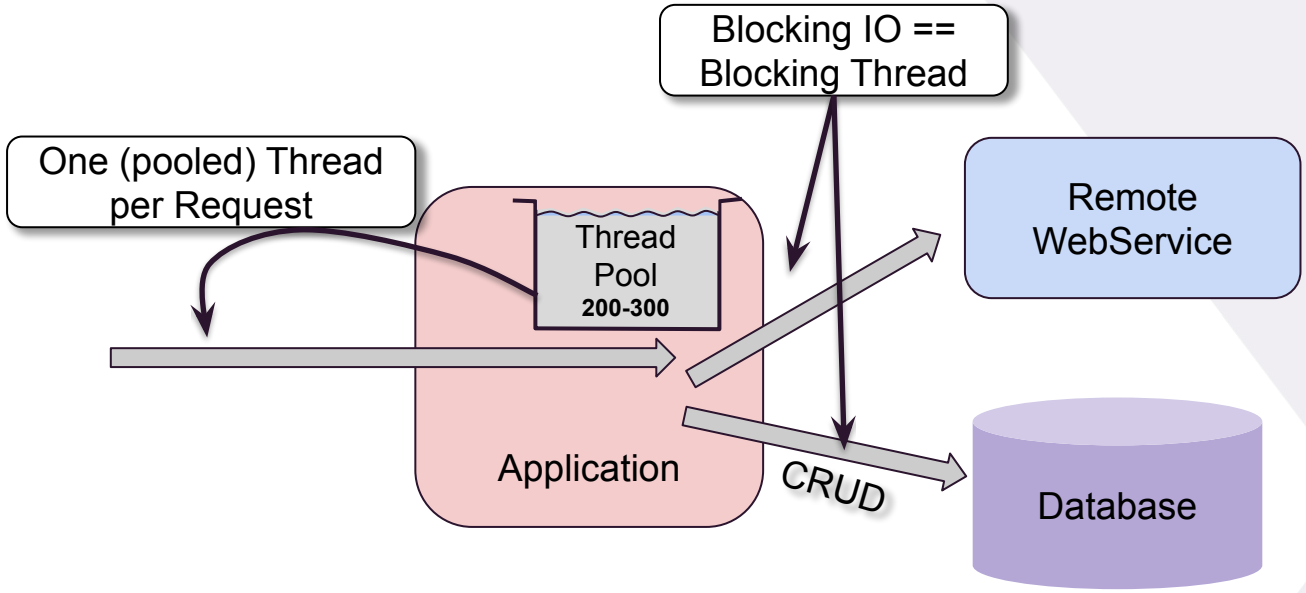
- Coroutines & Spring Boot: Introduction
- Coroutines & Spring Boot: Reactive DB Access & REST calls with Coroutines
- Coroutines & Spring Boot: Reactive Controllers with Coroutines

- Coroutines: Flow
- Coroutines & Spring Boot: Reactive Stream Controllers with Flow

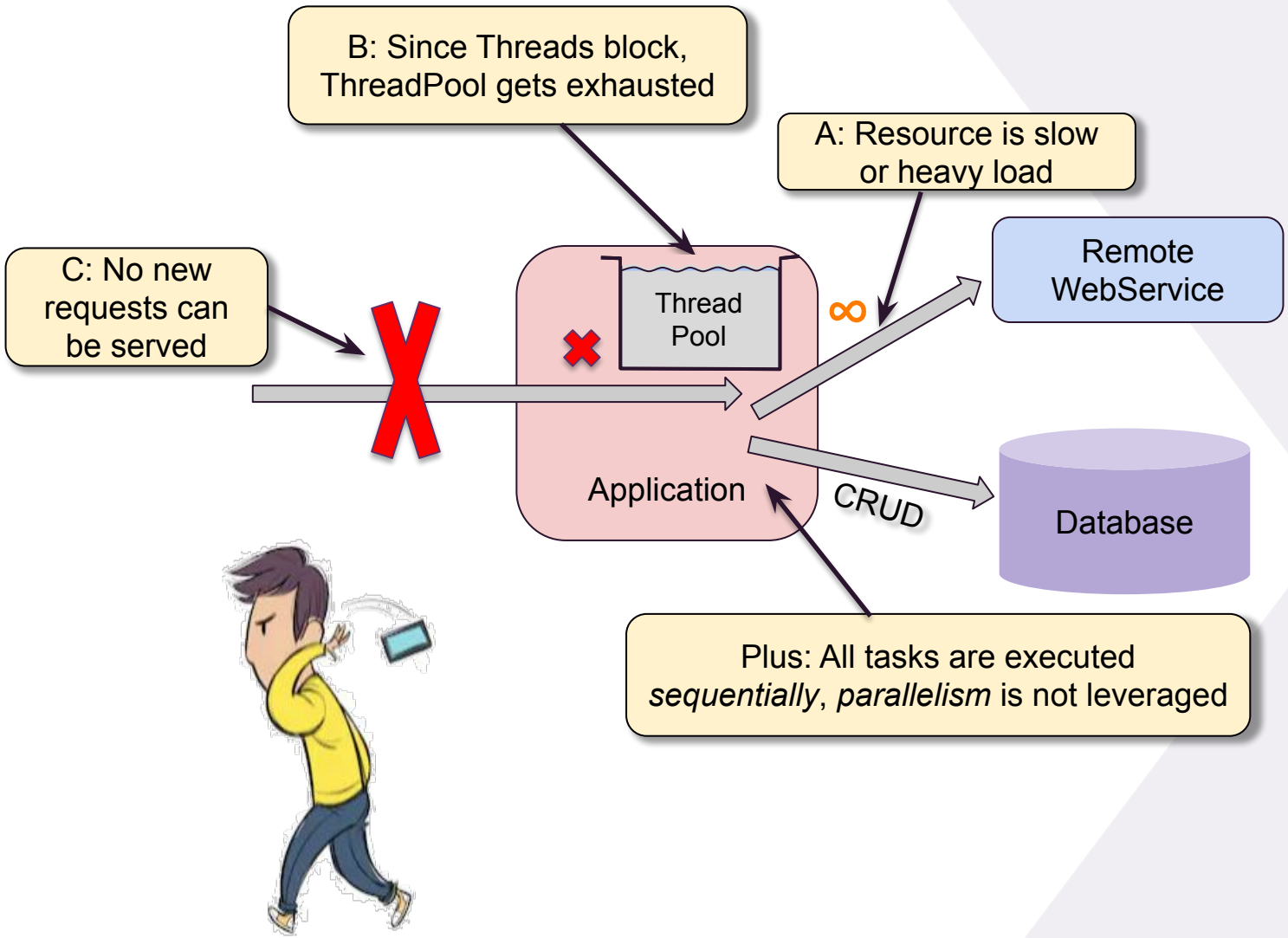
- Virtual Threads &| Coroutines



# Blocking Application Architectures

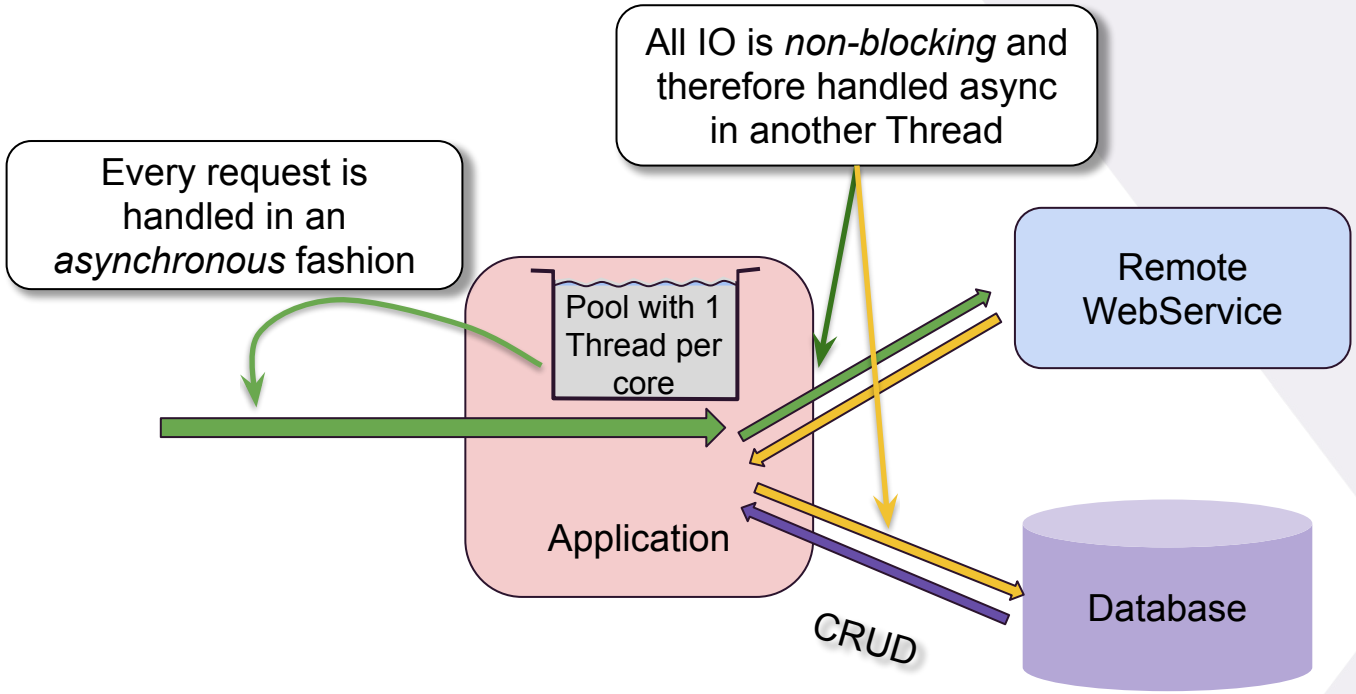


# Blocking Application Architectures

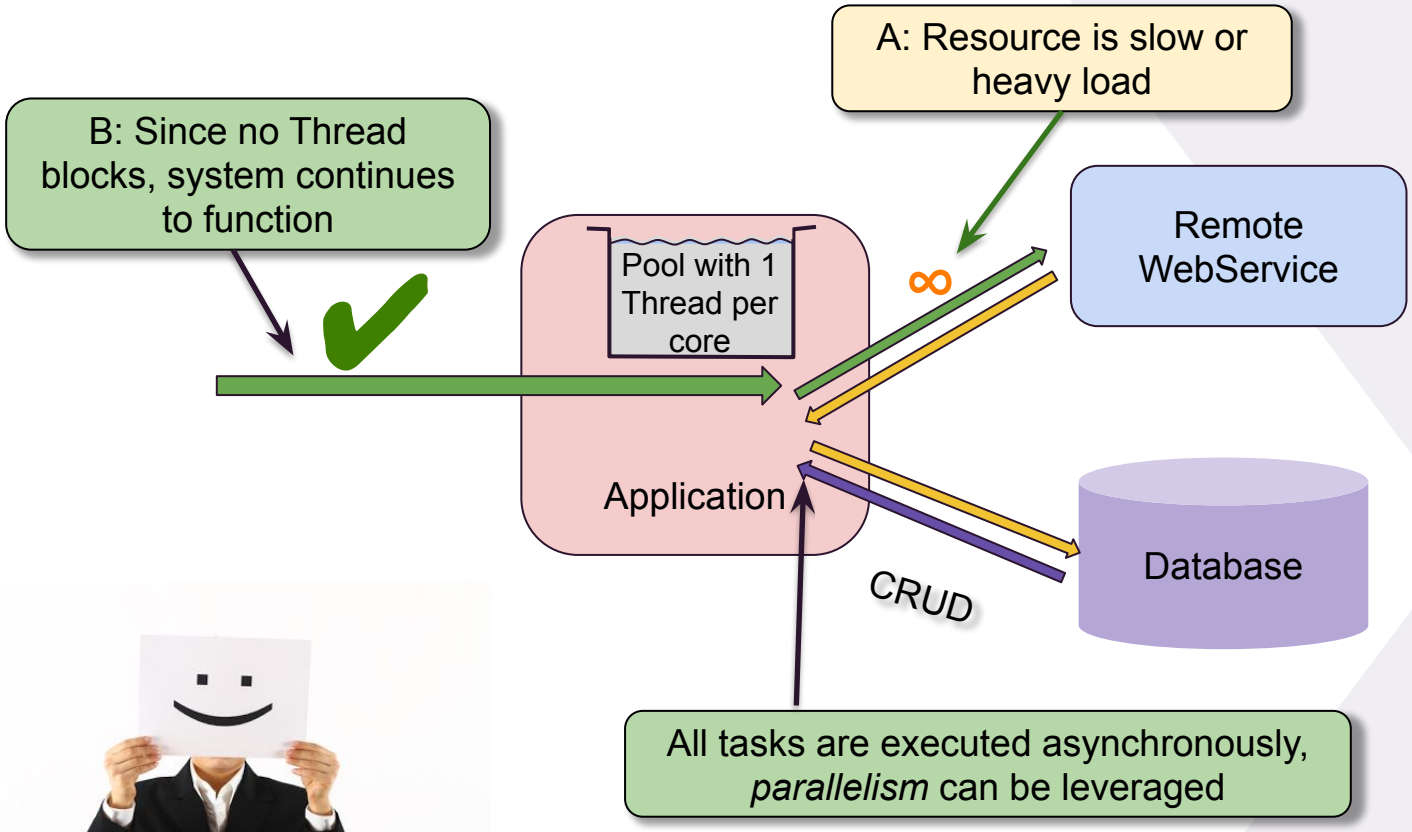



REACTIVE PROGRAMMING
TO THE RESCUE?
imgflip.com

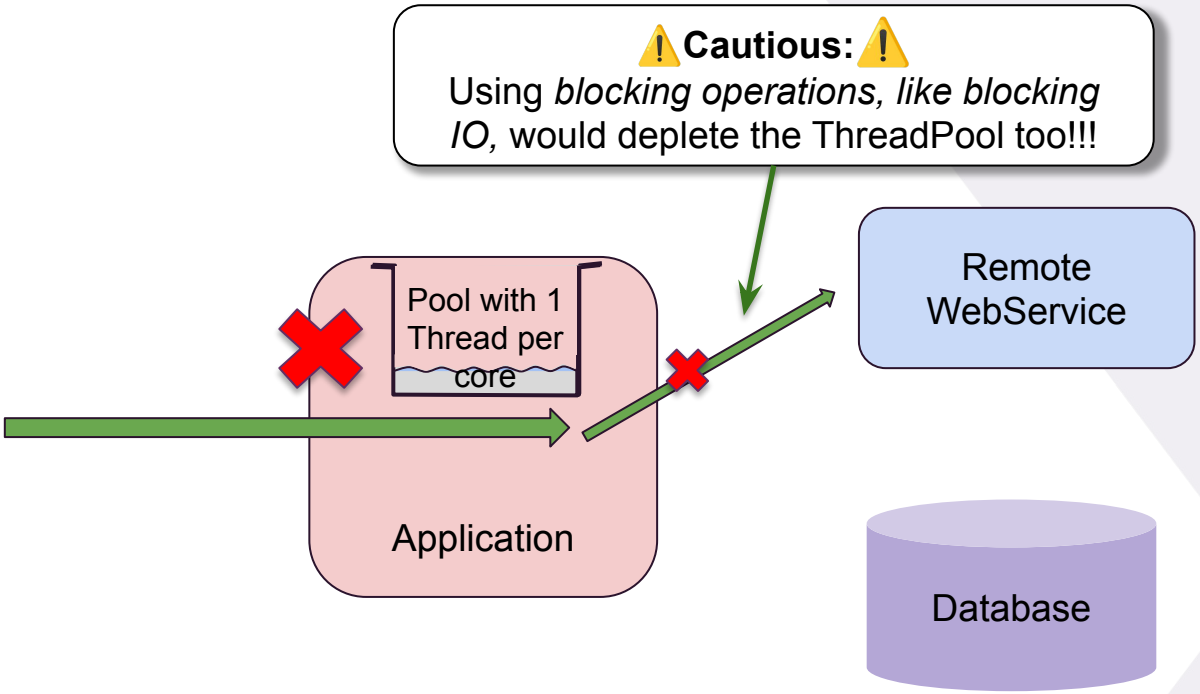
# Non-blocking Application Architectures



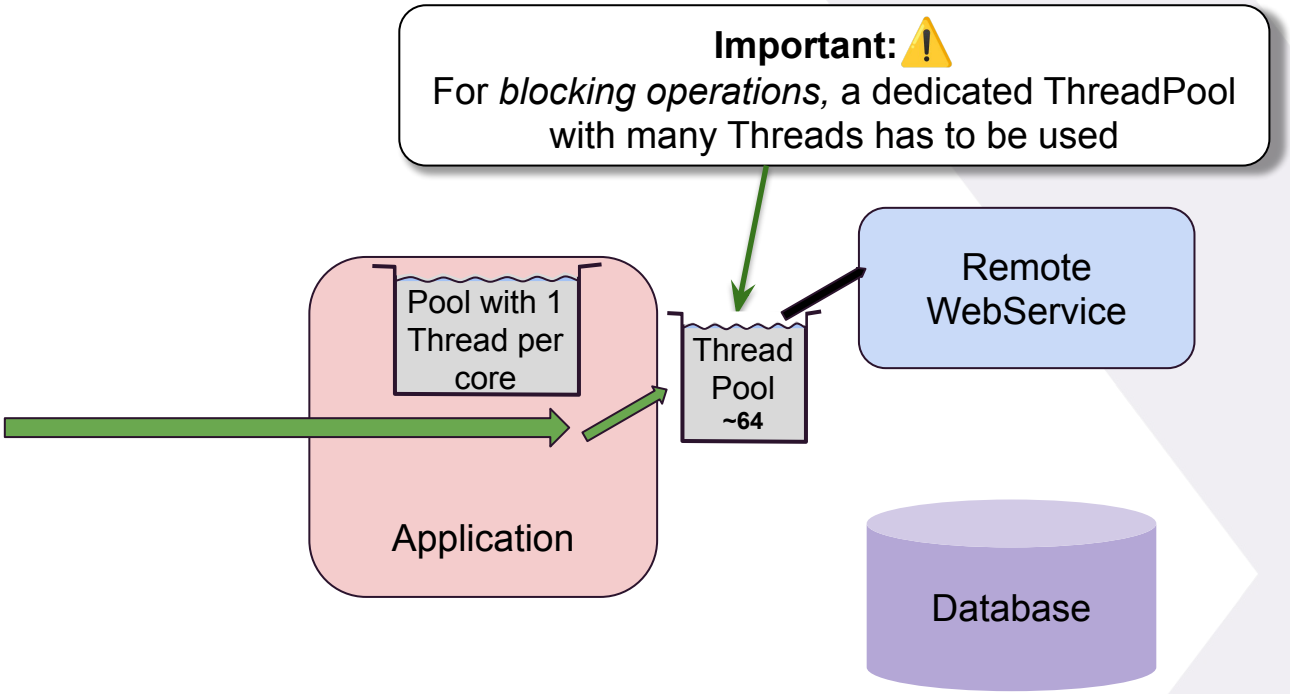
# Reactive Application Architectures



# Non-blocking/Reactive Application Architectures



# Non-blocking/Reactive Application Architectures



# Benefits of Reactive Application Architectures

- **Faster processing** times if the use-case allows parallel processing
- **Resilience** - a slow resource won't affect the rest of the application
- **Responsiveness** - replies can always be sent, no matter how slow / malfunctioning a resource
- **Considerably less memory** (~ 1 Thread per core for reactive vs hundreds in blocking applications)

# Reactive - the answer to all problems?

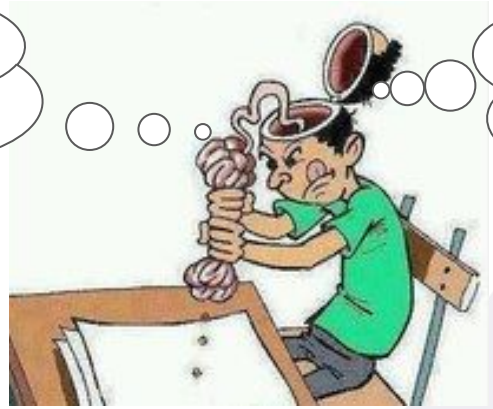
CompletableFutures, JavaRX, Reactor etc. offer *building blocks* for *async programming*.

```
fun randomAvatar(): Mono<URL> = ...//some remote async (non-blocking) call
fun verifyEmail(email:String): Mono<Boolean> = ...//some remote async call
fun save(user: User): Mono<Long> = ... //some async (non-blocking) database call
```

Since the async building blocks are libraries - and not *language features* - they force you to tightly bind your code to their API and abstractions.

```
val avatarMono = randomAvatar()
val verifyEmailMono = verifyEmail("joe@home.nl")
Mono.zip(avatarMono, verifyEmailMono).flatMap { (url, emailVerified) ->
   save(User(email = "joe@home.nl", avatarUrl = url, emailOk = emailVerified))
}.doFinally { user -> println("Saved $user with: ${user.id}") }
```

The ***business intent*** of my code gets **lost** in all the 'combinator jungle'



...and this stuff is *quite hard* **to learn** and **handle correctly** too *

* https://medium.com/jeroen-rosenberg/10-pitfalls-in-reactive-programming-de5fe042dfc6

# Reactive - the answer to all problems?

CompletableFutures, JavaRX, Reactor etc. offer *building blocks* for *async programming*.

```
fun randomAvatar(): Mono<URL> = ...//some remote async (non-blocking) call
fun verifyEmail(email:String): Mono<Boolean> = ...//some remote async call
fun save(user: User): Mono<Long> = ... //some async (non-blocking) database call
```

Since the async building blocks are libraries - and not *language features* - they allow you to tightly bind your code to their abstractions.

Price for
- Resource Efficiency (Non-Blocking IO)
- Little Parallelism

```
val avatarMono = randomAvatar()
val verifyEmailMono = verifyEmail("joe@home.
Mono.zip(avatarMono, verifyEmailMono).flatMa
                                    mailVerified) ->
   save(User(email = "joe@home.nl", avatarUrl = url, emailOk = emailVerified))
}.doFinally { user -> println("Saved $user with: ${user.id}") }
```

The ***business intent*** of my code gets **lost** in all the 'combinator jungle'

...and this stuff is *quite hard* **to learn** and **handle correctly** too *

* https://medium.com/jeroen-rosenberg/10-pitfalls-in-reactive-programming-de5fe042dfc6

# Reactive Programming to the rescue?

**yes and no:**

*reactive gets the job done but:*
*the price you pay in terms of complexity is enormous*

# The real answer? Coroutines & Spring Boot

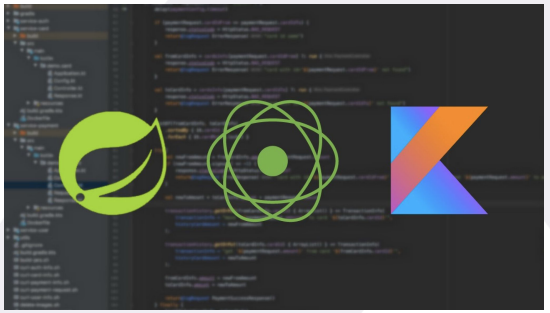
# Agenda

- Coroutines Basics: Structured Concurrency
- Coroutines Basics: Contexts + Propagation

- Coroutines & Spring Boot: Introduction
- Coroutines & Spring Boot: Reactive DB Access & REST calls with Coroutines
- Coroutines & Spring Boot: Reactive Controllers with Coroutines

- Coroutines: Flow
- Coroutines & Spring Boot: Reactive Stream Controllers with Flow

- Virtual Threads &| Coroutines



# Coroutines the wrong way in Spring Boot

```
@GetMapping("/users")
@ResponseBody
@Transactional
fun storeUser(@RequestBody user:User): User {
    val avatarUrl  = randomAvatar() //assuming a remote blocking call
    val validEmail = verifyEmail(user.email) //assuming a remote blocking call
    if(!validEmail) throw InvalidEmailException("Invalid email")
    return save(user.copy(avatar = avatarUrl)) //some blocking database call
}
```

spring MVC

The two remote calls I could actually do them in parallel…

I heard of these *'Coroutines'*, let's see if they can help

# Coroutines the wrong way in Spring Boot

Often developers start using Coroutines in Spring-web applications by introducing `runBlocking`.
*Why is that a bad idea?*

```
@GetMapping("/users")
@ResponseBody
@Transactional
fun storeUser(@RequestBody user:User): User = runBlocking {
    val avatarUrl  = async { randomAvatar()  //assuming a blocking call }
    val validEmail = async { verifyEmail() //assuming a blocking call }
    if(!validEmail.await()) throw InvalidEmailException("Invalid email")
    save(user.copy(avatar = avatarUrl.await()))
}
```

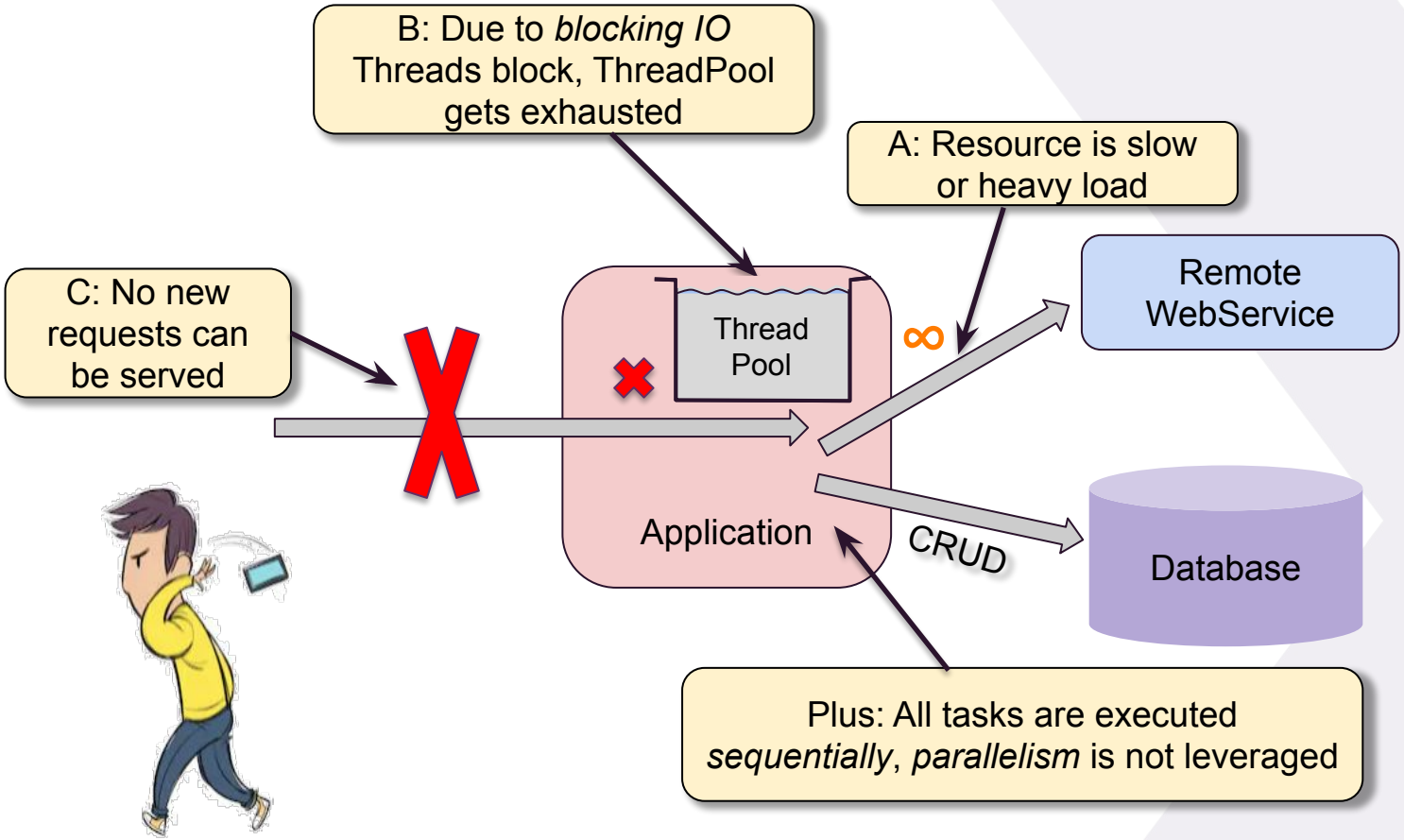
# Coroutines the wrong way in Spring Boot

So should you consider this approach, ***always*** use `runBlocking` in combination with `Dispatchers.IO`. Problem solved?

```
@GetMapping("/users")
@ResponseBody
@Transactional
fun storeUser(@RequestBody user:User): User = runBlocking(Dispatchers.IO) {
    val avatarUrl  = async { randomAvatar()  //assuming a blocking call }
    val validEmail = async { verifyEmail() //assuming a blocking call }
    if(!validEmail) throw InvalidEmailException("Invalid email")
    save(user.copy(avatar = avatarUrl))
}
```

# Coroutines the wrong way in Spring Boot

Not really, because you still face the discussed shortcomings of a blocking application architecture.



# Coroutines the wrong way in Spring Boot

…and there is more evil here than we see at first glance:

When using `Dispatchers.IO`, a different Thread than the Servlet Thread executes the Coroutines. So we lose all the state bound to the Servlet Thread like: Transaction, SecurityContext, MDC, etc.

```
@GetMapping("/users")
@ResponseBody
@Transactional
fun storeUser(@RequestBody user:User): User = runBlocking(Dispatchers.IO) {
    val avatarUrl  = async { randomAvatar()  //assuming a blocking call }
    val validEmail = async { verifyEmail() //assuming a blocking call }
    if(!validEmail) throw InvalidEmailException("Invalid email")
    save(user.copy(avatar = avatarUrl))
}
```

# Coroutines the wrong way in Spring Boot

…and there is more evil here than we see at first glance:

Context propagation could be mitigated by providing (custom) `ThreadContextElement` implementations to propagate the above contexts, like the `MDCContext()`, `~TransactionContext()`, `~SecurityContext()` etc.

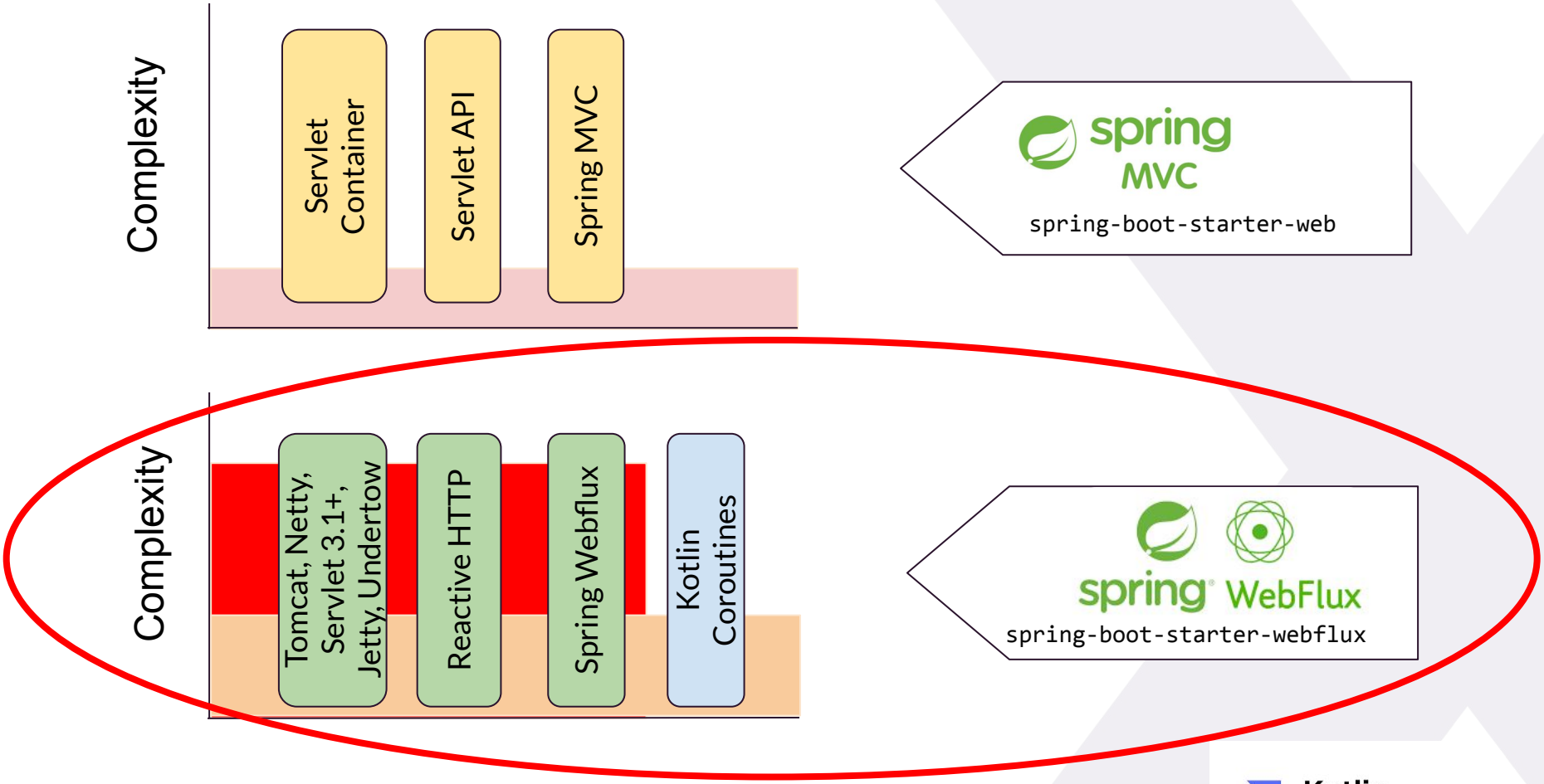
```
@GetMapping("/users")
@ResponseBody
@Transactional
fun storeUser(@RequestBody user:User): User = runBlocking(Dispatchers.IO + MDCContext() +
                                                          TransactionContext() + …) {
    val avatarUrl  = async { randomAvatar()  //assuming a blocking call }
    val validEmail = async { verifyEmail() //assuming a blocking call }
    if(!validEmail) throw InvalidEmailException("Invalid email")
    save(user.copy(avatar = avatarUrl))
}
```

- However, this can be dangerous too: e.g. doing parallel database operations using `async`/`await` causes Threading issues, because standard Transactions are not Thread-Safe.

**So, how does the right way look like?**

# Coroutines the right way in Spring Boot

Use the winning formula: Spring Boot with WebFlux with Coroutines!



# Coroutines the right way in Spring Boot

Use the winning formula: Spring Boot with Reactor and Coroutines!

Not truly reactive:
Filters, Servlets are synchronous, no back pressure, limited servers

```
spring-boot-starter-web
spring-boot-starter-webflux
```

spring MVC WebFlux

Truly reactive:
Filters, Servlets are async, back pressure support and wide variety of servers

```
spring-boot-starter-webflux
```

spring WebFlux

Kotlin Coroutines

# Coroutines the right way in Spring Boot

Besides Spring's webflux dependency the following are needed to enable Coroutine support in Spring:

```
dependencies {
implementation("org.springframework.boot:spring-boot-starter-webflux:${spring.boot.version}")
     implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core-jvm:${kotlinx.version}")
     implementation("org.jetbrains.kotlinx:kotlinx-coroutines-reactor:${kotlinx.version}")
}
```

Coroutines work on top of WebFlux. The above dependencies provide the glue code that lifts the WebFlux concurrency primitives into the Coroutine space, **which makes reactive programming so much easier - as you will see.**

spring BOOT

Kotlin Coroutines

# From WebFlux to Coroutines: Mono

Reactor uses the following primitive to handle single asynchronous values:

`Mono<T>`

- Handles a value or `Exception` that eventually becomes available
- Is similar to a `CompletableFuture<T>` except that it is initialized lazily

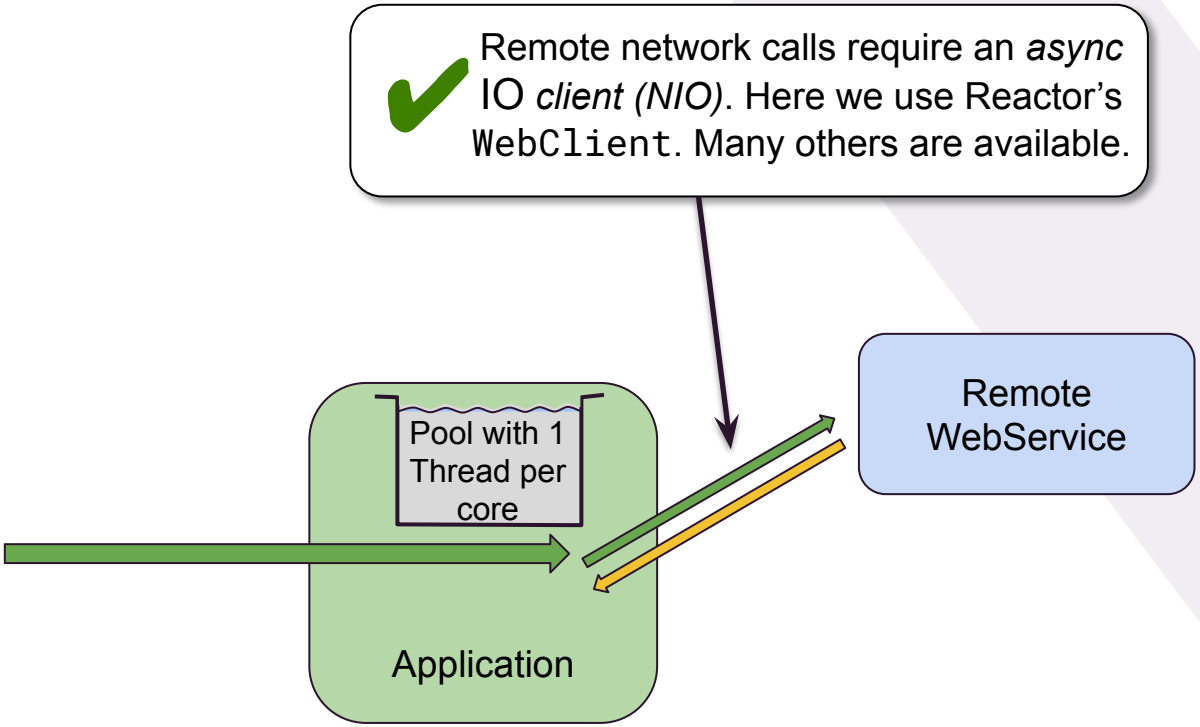
When using Coroutines, `Mono<T>` literally *disappears*.

- Functions that returned a `Mono<T>` become a `suspend` functions that return `T` or throw an `Exception`.

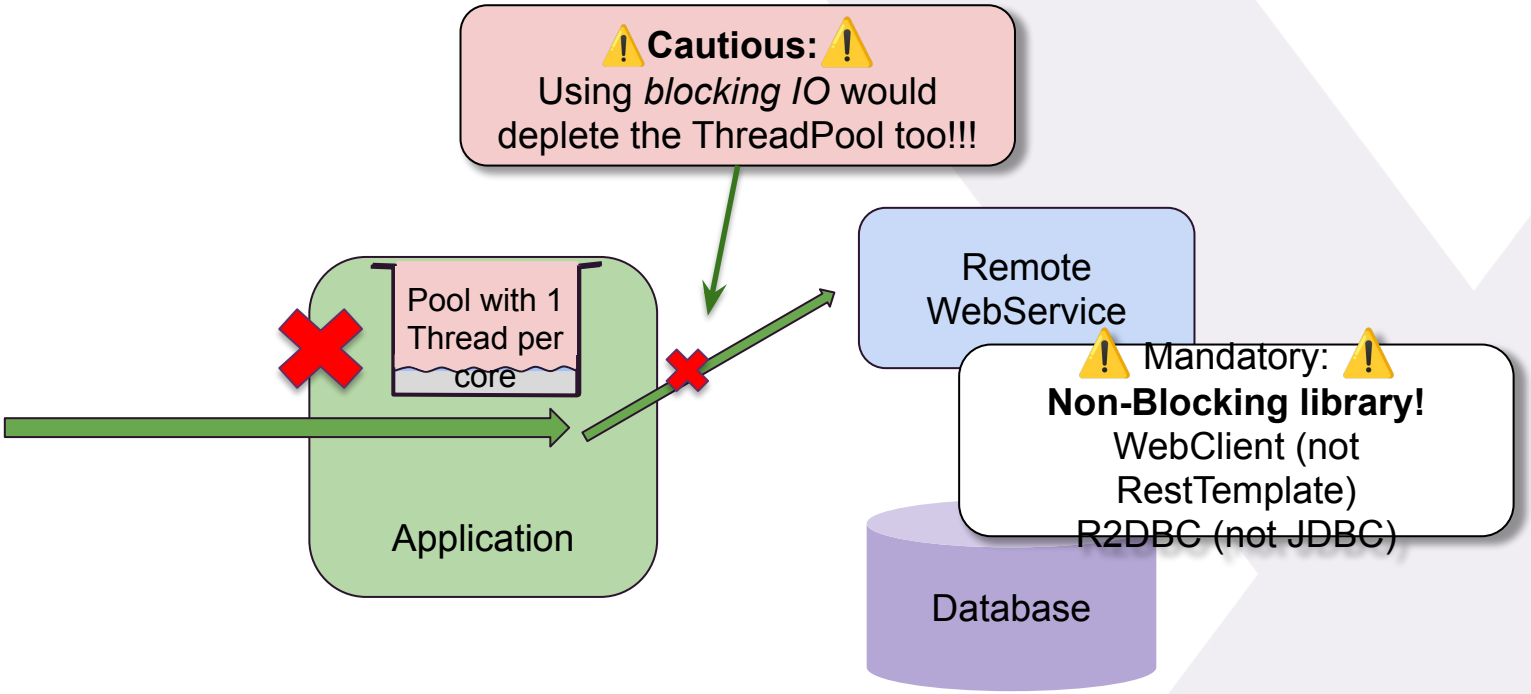
So with Coroutines we *get completely rid of the `Mono<T>` abstraction* and enjoy 'normal' function signatures having the `suspend` keyword.



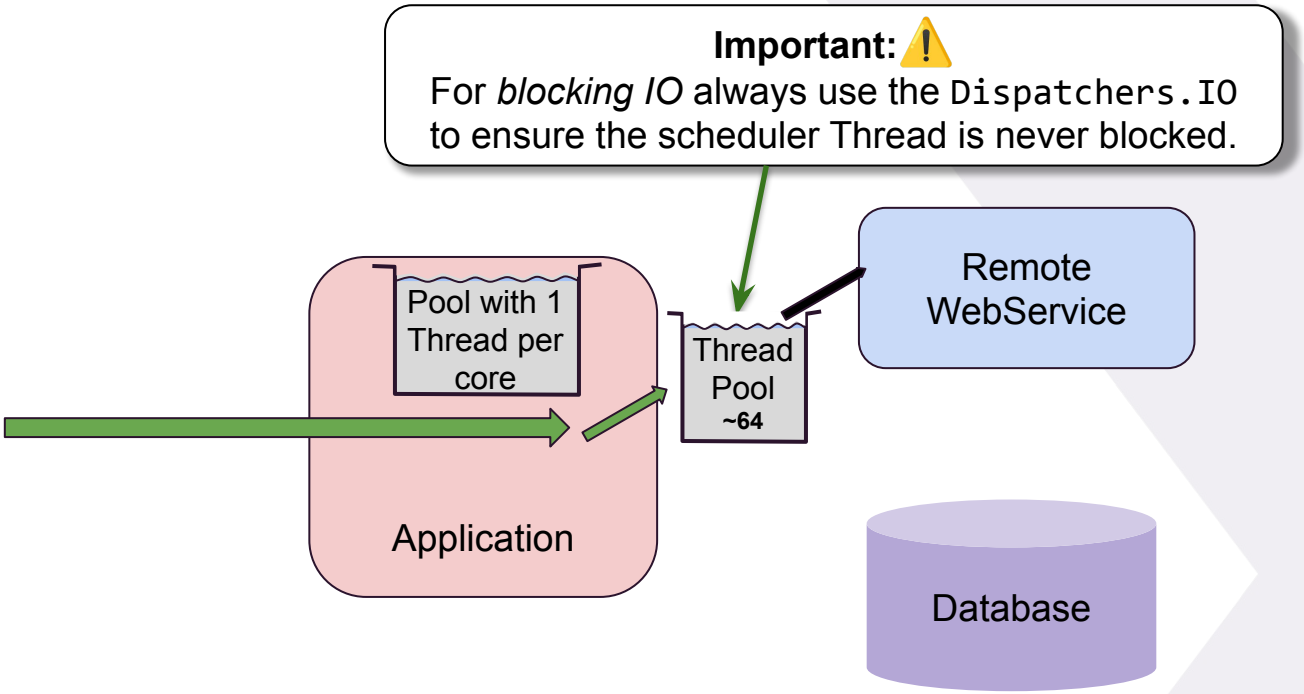
# Calling Remote Services in reactive Spring Boot



# Calling Remote Services in reactive Spring Boot



# Calling Remote Services in reactive Spring Boot



# Remote Service Calls with Reactor & Coroutines

Spring's `WebClient` used for remote non-blocking REST calls, is based on `Mono<T>`

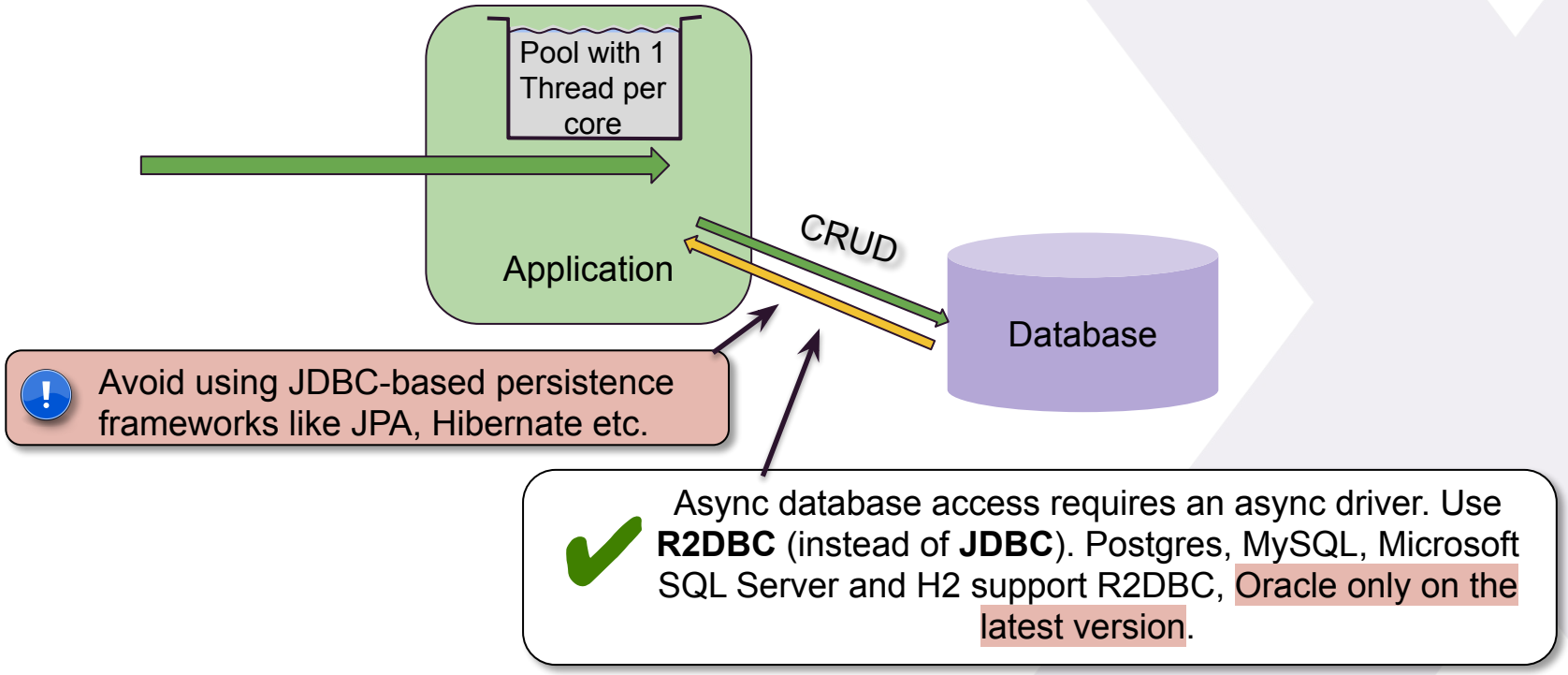
```java
@Component
public class AvatarService {

    public Mono<URL> randomAvatar() {
        return WebClient.create("http://<host>")
                        .get()
                        .uri("/avatar")
                        .retrieve()
}
```


```kotlin
import org.springframework.web.reactive.function.client.awaitBody

@Component
class AvatarService {

    suspend fun randomAvatar(): URL = WebClient.create("http://<host>")
                    .get()
                    .uri("/avatar")
                    .retrieve()
                    .awaitBody<URL>()
```


With Coroutines simply mark remote service calls methods with `suspend`.

...and use one of the 'glue methods' `await...` that turn a `Mono<T>` into a suspended call. And gone is the `Mono<T>` abstraction 😃! Important: Never use `.retrieve().block()`!

# Database Access in reactive SpringBoot



# Database Access with Reactor & Coroutines

Spring's reactive repositories rely on `Mono<T>`'s for single result repository calls.

```java
@Repository
interface UserDao extends ReactiveCrudRepository<User, Long> {

    public Mono<User> findByUserName(String userName);
}
```


With Coroutines extend repositories from `org.springframework.data.repository.kotlin.CoroutineCrudRepository`

```kotlin
@Repository
interface UserDao : CoroutineCrudRepository<User, Long> {

    suspend fun findByUserName(userName: String) : User?

}
```

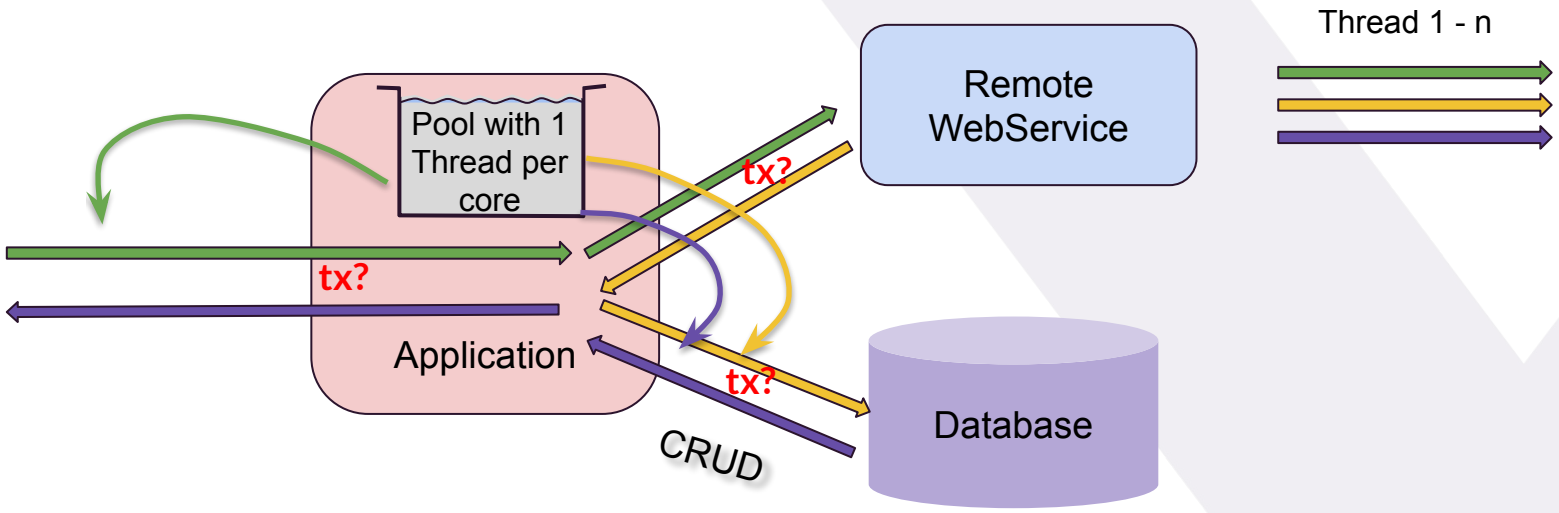
Kotlin

Mark additional queries with `suspend`.

To define queries use spring-data's common naming syntax or `@Query` annotations

We can also make return types safer by introducing nullability.

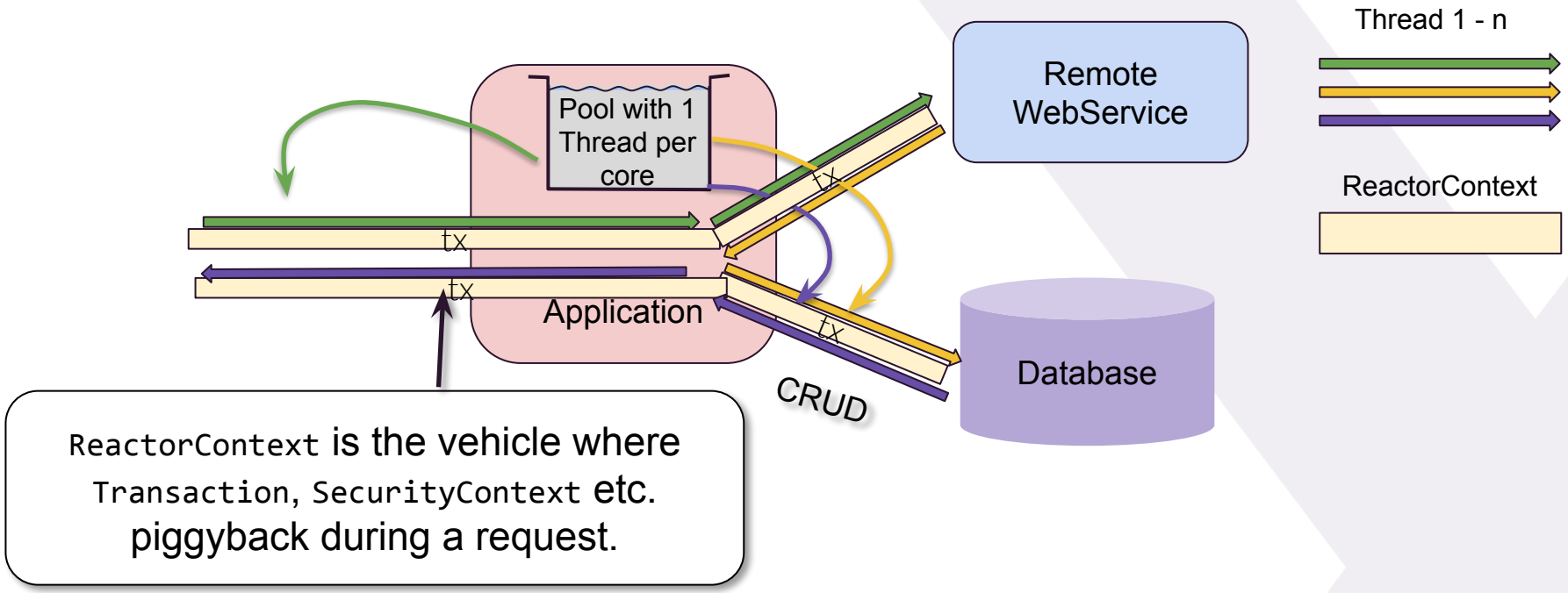
# How about context propagation?




In the blocking architecture we saw that `Transaction`, `SecurityContext` etc. are bound to the `Thread` that handles the request via `ThreadLocal`.

But in the non-blocking approach, `ThreadLocals` won't work because (potentially) ***many*** `Threads` take part in handling the request. How are these contexts propagated then?

# …by means of the ReactorContext!




The `ReactorContext` is a `Map`-like data structure that is made available to all parts of a *reactive pipeline* (here a request) by means of the `Subscriber`.

As such, state in the `ReactorContext` is `Thread` agnostic, unlike `ThreadLocals`,which are always bound to a `Thread`.

**Important**: `ThreadLocal` based frameworks like SLF4J's MDC will only work in 'pure' webflux applications using the context-propagation library.

# How does this fit into Coroutines?

The good news is that, conceptually, a `CoroutineContext` and a `ReactorContext` are identical.

The even better news is, that, by having `org.jetbrains.kotlinx:kotlinx-coroutines-reactor:<version>` on your classpath, the `ReactorContext` is *automatically* propagated to the `CoroutineContext`.

This is why `Transactions` just work and all contexts (`SecurityContext` etc.) are available in Coroutines when combined with Reactor.

The `ReactorContext` can be looked up in the `coroutineContext` with the key: `ReactorContext`.

```
@Component
class UserService {

  suspend fun myServiceMethod() : User = coroutineScope { this:CoroutineScope
     val reactorContext = this.coroutineContext[ReactorContext]?.context ?:
                    throw IllegalStateException("ReactorContext must be present")

     val exchange  = reactorContext.get<ServerWebExchange>(
                              ServerWebExchangeContextFilter.EXCHANGE_CONTEXT_ATTRIBUTE)
     ...
}
```

# Exercise: Remote Service Calls with Reactor & Coroutines

- Slides: https://bit.ly/kotlinconf-springboot-workshop-slides-2026
- Sources: https://bit.ly/kotlinconf-springboot-workshop-labs-2026

- Open: StockRepository, ExchangeService and Coroutines01ServiceRepositoryTest
- **Solve Exercise: A-B**
  - `Coroutines01ServiceRepositoryTest.kt`: here you find the instructions of all exercises.
  - `StockRepository.kt`: in this file you have to provide an implementation for Challenge 3 - Part 1 - exercise A
  - `ExchangeService.kt`: in this file you have to provide an implementation for Challenge 3 - Part 1 - exercise B

You have completed the challenge if all tests succeed in `Coroutines01ServiceRespositoryTest`.

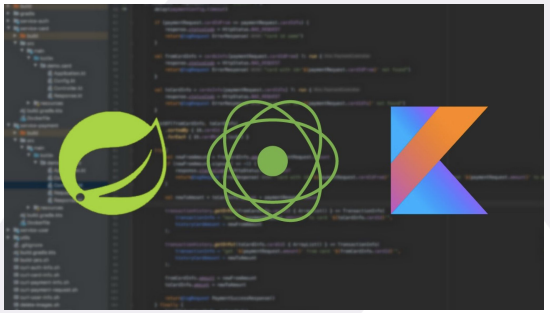
# Agenda

- Coroutines Basics: Structured Concurrency
- Coroutines Basics: Contexts + Propagation

- Coroutines & Spring Boot: Introduction
- Coroutines & Spring Boot: Reactive DB Access & REST calls with Coroutines
- Coroutines & Spring Boot: Reactive Controllers with Coroutines

- Coroutines: Flow
- Coroutines & Spring Boot: Reactive Stream Controllers with Flow

- Virtual Threads &| Coroutines



# Controllers with Reactor & Coroutines

```java
@RestController
public class DummyController {

   @GetMapping("/delayed")
   @ResponseBody
    public Mono<String> nonBlockingDelay() {
      return Mono.just("Sorry for the delay of 1 second")
         .delayElement(Duration.of(1, ChronoUnit.SECONDS));
}
```

To make an endpoint reactive in Java you have to return a `Mono<T>`.

With Coroutines simply mark a controller method `suspend`. There is no need to use a CoroutineBuilder, Spring does that for us.

```kotlin
@RestController
class DummyController() {

   @GetMapping("/delayed")
   @ResponseBody
    suspend fun nonBlockingDelay(): String {
        delay(1000)
        return "Sorry for the delay of 1 second"
}
```

# Drawbacks of reactive abstractions...

Reactive abstractions like `Mono's` are *viral* and will dominate your whole codebase, blurring the business intent of your code significantly. Also programming the 'Mono way' takes quite some time to master *

```
@RestController
public class PersonController {

   @PostMapping("/persons")
   @ResponseBody
    public Mono<User> storeUser(@RequestBody User user) {
     Mono<Avatar> avatarMono = avatarService.randomAvatar();
     Mono<Boolean> validEmailMono = emailService.verifyEmail(user.getEmail());
      return Mono.zip(avatarMono, validEmailMono).flatMap(tuple ->
       if(!tuple.getT2()) //what is getT2()? It's the validEmail Boolean...
           Mono.error(new InvalidEmailException("Invalid Email"));
       else
          personRepo.save(UserBuilder.from(user)
                                     .withEmail(user.getEmail())
                                     .withAvatar(tuple.getT1()));
      );
    }
```

# ...are gone with Coroutines

With Coroutines there is no additional reactive abstraction we have to deal with. We can stick to normal method signatures, sequential programming flow and common language constructs like `if/else`, `when`, `throw`, `try/catch` etc. to express business logic, which captures our intent concisely. The only thing we need is the `suspend` keyword.

Kotlin

```
@RestController
class PersonController {

    @GetMapping("/persons")
    @ResponseBody
    suspend fun storeUser(@RequestBody user:User): User {
        val validEmail = emailService.verifyEmail()
        if(!validEmail) throw InvalidEmailException("Invalid email")
        val avatarUrl = avatarService.randomAvatar()
        return personRepo.save(user.copy(avatar = avatarUrl))
}
```

# async on demand

We can also make use of parallelism using `async/await`. We only have to declare `coroutineScope` helper method that implicitly exposes the `CoroutineScope` of the `suspend` method, which is needed to create an `async` Coroutine.

```kotlin
@RestController
class PersonController {

    @GetMapping("/persons")
    @ResponseBody
    suspend fun storeUser(@RequestBody user:User): User = coroutineScope {
        val avatarUrl = async { avatarService.randomAvatar() }
        val validEmail = async { emailService.verifyEmail() }
        if(!validEmail.await()) throw InvalidEmailException("Invalid email")
        personRepo.save(user.copy(avatar = avatarUrl.await()))
}
```

# Testing Reactive Endpoints

`WebTestClient` needs to be used to test reactive endpoints. `MockMvc` cannot be used. `WebTestClient` can also be used to test REST endpoints remotely.

```kotlin
@SpringBootTest
@ExtendWith(SpringExtension::class)
@AutoConfigureWebTestClient
class PersonControllerTest @Autowired constructor(
                val webTestClient: WebTestClient) {
    @Test
    fun `GET to persons should return list of persons`(): Unit  {

        webTestClient.get().uri("/persons").exchange().apply {
            expectStatus().isOk
            expectBody<List<Person>>()
                    .returnResult().responseBody?.shouldNotBeEmpty()
        }
    }
}
```

`WebTestClient` also offers a test DSL to create REST requests and verify responses.

# Exercise: Controllers & Coroutines

- Open: StockTraderController and Coroutines02ControllerTest
- **Solve Exercise: A-D**
  - `StockTraderController.kt`: find the instructions for exercises named Challenge 3 - Part 2 A-D.
  - `Coroutines02ControllerTest.kt`: make all tests succeed.

You have completed the challenge if all tests succeed in `Coroutines02ControllerTest`.

# Agenda

- Coroutines Basics: Structured Concurrency
- Coroutines Basics: Contexts + Propagation

- Coroutines & Spring Boot: Introduction
- Coroutines & Spring Boot: Reactive DB Access & REST calls with Coroutines
- Coroutines & Spring Boot: Reactive Controllers with Coroutines

- Coroutines: Flow
- Coroutines & Spring Boot: Reactive Stream Controllers with Flow

- Virtual Threads &| Coroutines

# Processing a Stream of Values

What I'm still missing is a primitive to process **a stream of** - possibly infinite - **values over time**. Is there something like that?

# Go with the Flow

A `Flow<T>` represents a *non-blocking stream* of *data*. Similar like a sequence.

```
interface Flow<out T> {
  suspend fun collect(collector: FlowCollector<T>)
}
```

A builder method is used to created a Flow<T>:

```
fun <T> flow(block: suspend FlowCollector<T>.() -> Unit): Flow<T>
```

```
val helloFlow: Flow<String> = flow { this: FlowCollector<String>
    emit("~ Go with ")
    delay(1000L)
    emit("the Flow ~")
}
```

To emit a value, use the FlowCollector's `emit(…)`:

```
interface FlowCollector<in T> {
  suspend fun emit(value: T)
}
```

*Unlike* a sequence, a flow is *asynchronous* and allows *suspend functions* in its builder and operators

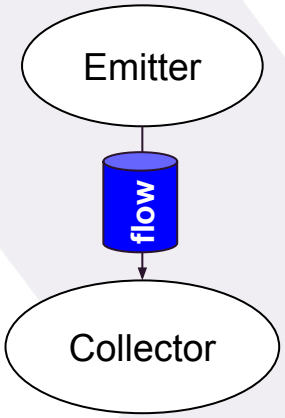
# Creating and consuming Flow

A builder method needs to be used to create a Flow

```
val helloFlow:Flow<String> = flow {
        emit("It's time to: ")
        emit("~ Go with ")
        delay(1000L)
        emit("the Flow ~")
    }
```

Emit data to the flow

Call suspend methods




Flows can be transformed using various *intermediate operators* like `map`, `filter` etc.

```
helloFlow.filter{ it.contains("~") }
         .map{ it.uppercase() }
         .collect{ print(it) }
```

*Terminal operators* collect all values emitted by the flow. They will also *activate* the flow code

1. What does the statement above print?
2. How long does it take (approximately)?
3. What happens if the statement is re-run again?

# Flows are sooo coool

Flow is a **cold** source. It is:
- not active *before* the call to *terminal operation*
- not active *after*
- releasing all resources before returning from the call

```
val helloFlow:Flow<String> = flow {
        (1..5).forEach{ emit("Hi $it") }
    }.onEach { println("On each $it") }
     .onStart { println("Starting flow") }
     .onCompletion { println("Flow completed")
}
```

As long as no terminal operator is called, nothing will happen

```
helloFlow.filter{ it.contains("~") }
```

Once a *terminal operator* is invoked, the flow is triggered

```
helloFlow.filter{ it.contains("~") }
         .toList()
```

# Flows are sooo  coool

Flow is a **cold**  source. It is:
- not active *before* the call to *terminal operation*
- not active *after*
- releasing all resources before returning from the call

```
val helloFlow:Flow<String> = flow {
       emit("It's time to: ")
       emit("~ Go with ")
       delay(1000L)
       emit("the Flow ~")
   }
```

```
helloFlow.filter{ it.contains("~") }
          .map{ it.uppercase() }
          .collect{ print(it) }
```

# Flow Lifecycle Methods

```
val helloFlow:Flow<String> = flow {
      (1..5).forEach{ emit("Hi $it") }
 }.onStart {
      println("Starting flow")
 }.onEach {

      println("On each $it")
 }.onCompletion {
      println("Flow completed")
 }
```

In every flow operator (e.g. `map`, `filter`, `on`… etc.) we can call `suspend` methods

# Flow Creation without the flow Builder

```
val helloFlow:Flow<String> = flow {
        (1..5).forEach{
             emit("Hi $it")
        }
   }
```

```
val helloFlow1:Flow<String> =
    flowOf("Hi 1", "Hi 2", "Hi 3", "Hi 4", "Hi 5")
```

`flowOf(...)` factory method

```
val helloFlow2:Flow<String> =
    (1..5).map{ "Hi $it" }.asFlow()
```

convert Collections to Flow with `asFlow()`

```
val helloFlow3:Flow<String> =
    generateSequence(seed = 1).map{it + 1}.asFlow()
```

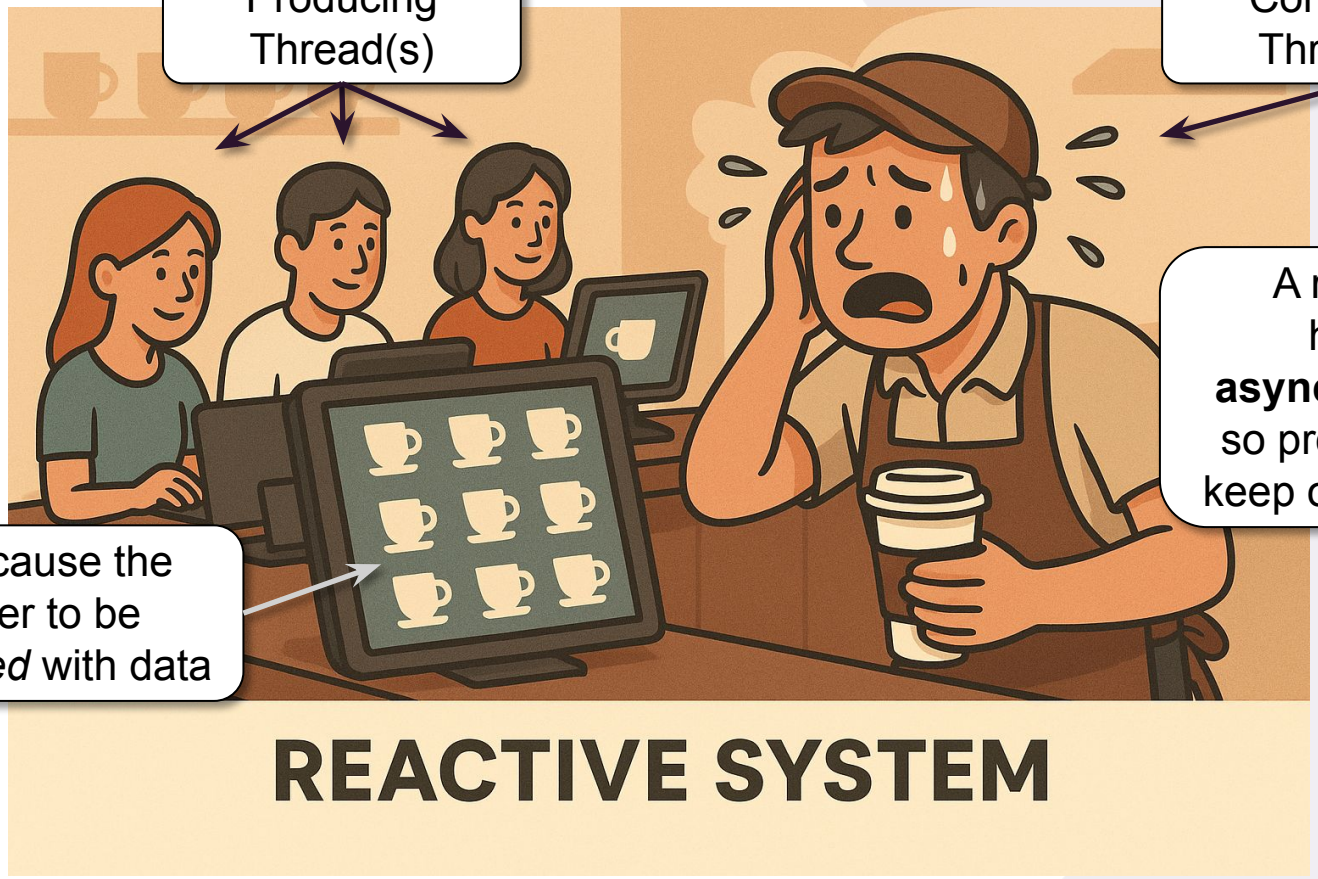
`generateSequence(...)` for creating an infinite flow

# A problem ‘blocking’ does not have:

A thread processes a request **synchronously**. As such it cannot be overwhelmed by new data.


Other customers are ‘blocked’.

# Reactive: risk of overwhelming the consumer

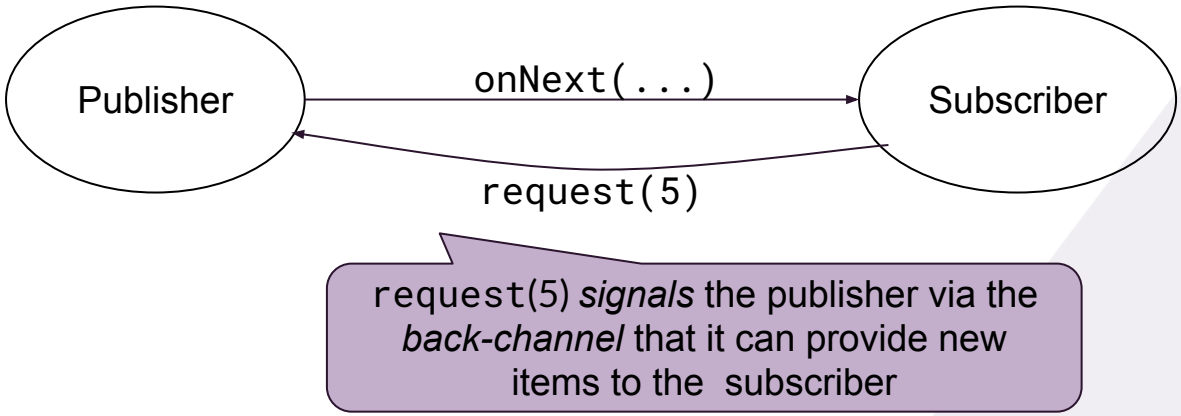


# Back-pressure

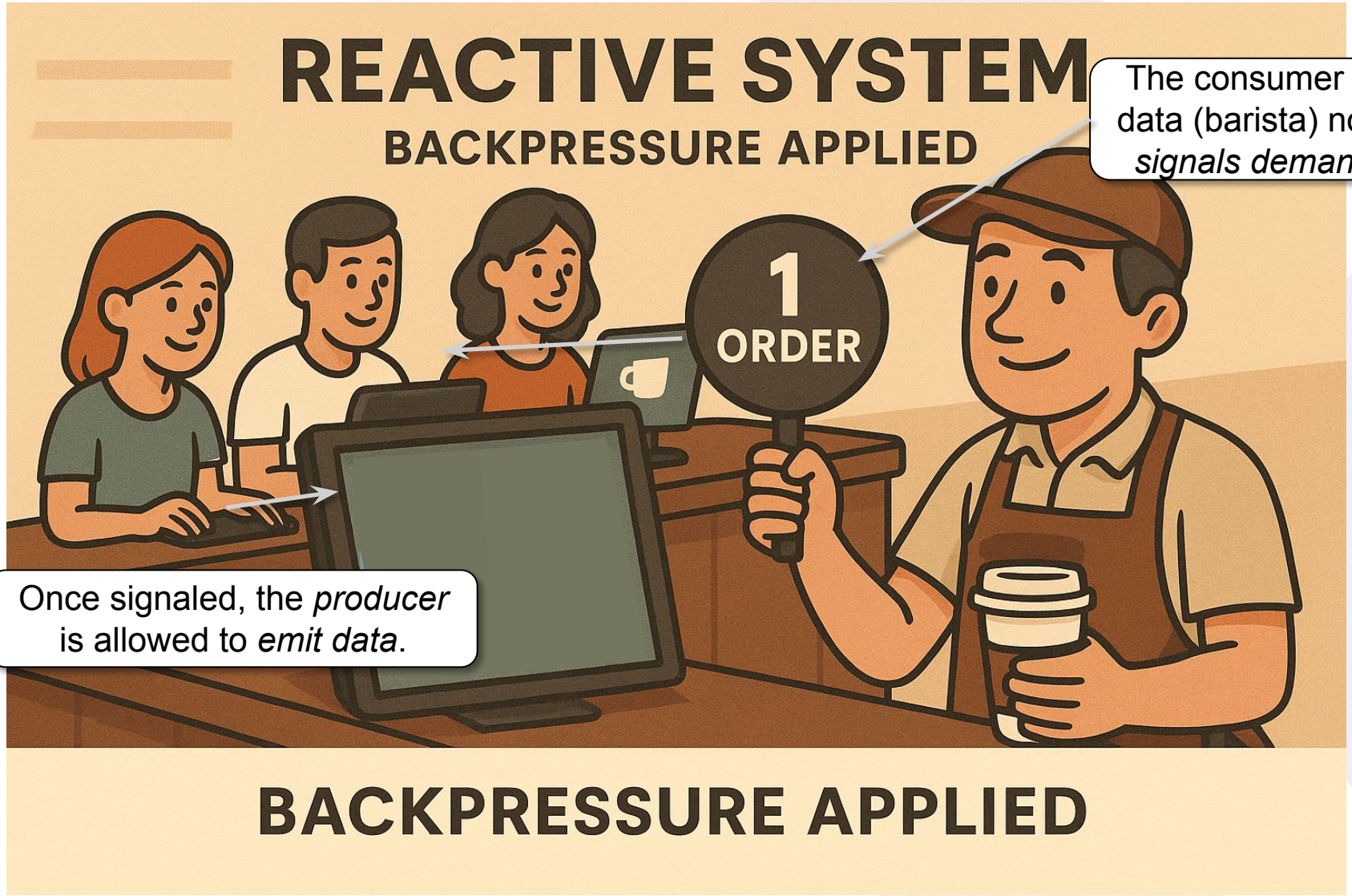
Back-pressure is defined as the ability of a *consumer* that cannot keep up with the incoming data to send a *signal* to the *producer* to slow down the rate of data elements.

In traditional reactive stream implementations like RxJava, Reactor etc. this *signaling* is achieved with a *back-channel* that *requests* more data.

This so called 'push-pull hybrid approach' leads to notoriously difficult implementations.



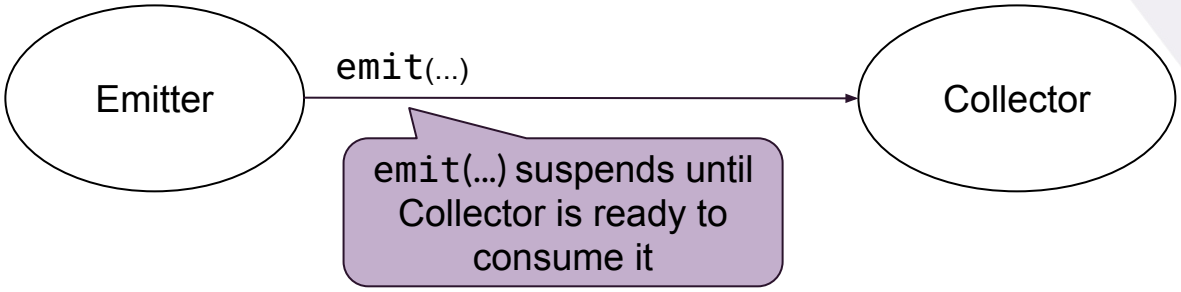
# Back-pressure prevents overwhelming consumer



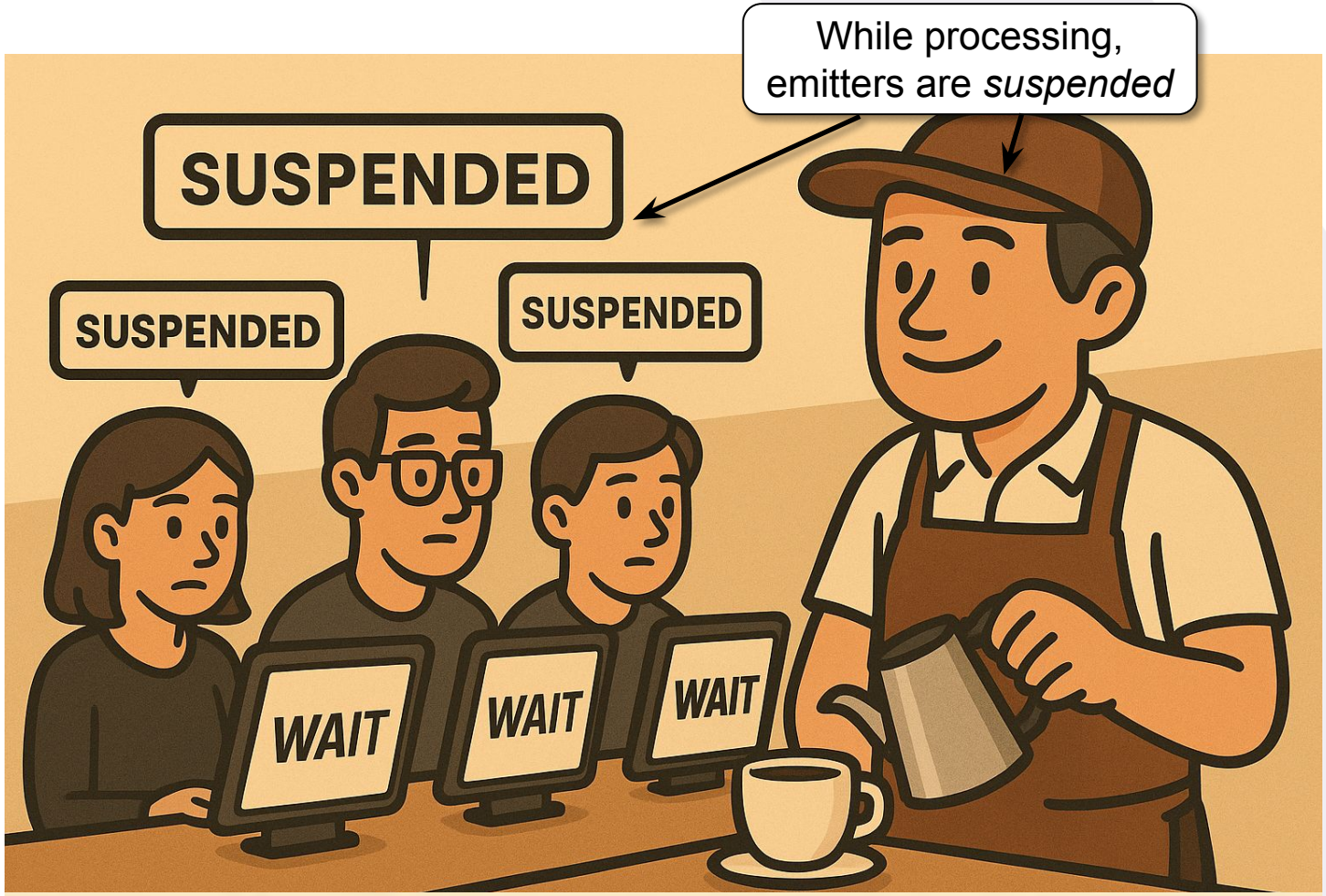
# Flow and Back-pressure

Flow requires no such complex mechanism because it relies on *suspend functions* that inherently support back-pressure:

When a consumer of a Flow is overwhelmed, it simply *suspends* the emitter and *resumes* it later when it is ready to accept more elements.



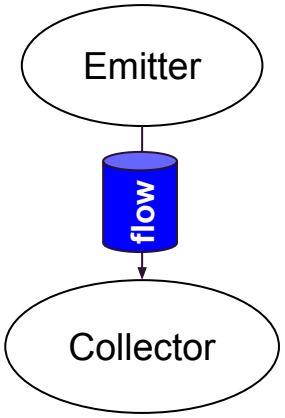
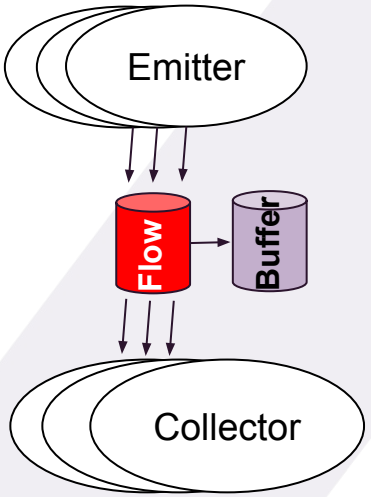

# Async processing yet Back-pressure by suspending




**AUTOMATIC BACKPRESSURE BY SUSPENDING**

# Hot Flows: StateFlow & SharedFlow

Besides the cold flow there are also hot flows in the form of `StateFlow` and `SharedFlow`:

| Flow type | cold `flow{...}` | hot `StateFlow` and `SharedFlow` |
|---|---|---|
| **Collectors** | One collector per flow | Multiple collectors per flow |
| **Producer** | Collecting (calling `collect {...}`) will trigger producer code - eventually `emit(...)` | Collecting (calling `collect {...}`) will not trigger producer code (`emit(...)`) |
| **Paradigm** |  |  |

# StateFlow in Action

`StateFlow` requires an *initial value*. It only can keep a *single value*. As such it works similar like a `ConflatedBroadcastChannel`.

```
runBlocking {
    val stateFlow = MutableStateFlow("Initial")
    launch {
        stateFlow.collect{ println("Turtle 1 meets $it") }
    }
    delay(100)
    stateFlow.emit("Rabbot")
    stateFlow.emit("Rabbot")
    delay(100)
    stateFlow.value = "Rabbit"
    launch {
        stateFlow.collect{ println("Turtle 2 meets $it") }
    }
}
```

Delay is required to consume initial value

Duplicates (*compared by value* rather than reference) will not be emitted but *skipped*.

`.value` allows accessing the current value without calling `collect{...}`. `.value = newValue` can be used as an alternative to `emit(newValue)`

# SharedFlow in Action

`SharedFlow` is a subclass of `StateFlow`. It requires no *initial value*. `replay` defines the amount items that are *buffered* for new collectors.

```
runBlocking {
    val sharedFlow = MutableSharedFlow<String>(replay = 1)
    launch {
        sharedFlow.collect{ println("Turtle 1 meets $it") }
    }
    sharedFlow.emit("Rabbyte")
    delay(100)
    sharedFlow.emit("Rabbot")
    launch {
        sharedFlow.collect{ println("Turtle 2 meets $it") }
    }
    delay(100)
    sharedFlow.emit("Rabbit")
}
```

Delay is required to consume emitted value. With `replay=1` it might be skipped by the collector if another value is emitted immediately after.

Same here

Run: MainKt

```
Turtle 1 meets Rabbyte
Turtle 1 meets Rabbot
Turtle 2 meets Rabbot
Turtle 1 meets Rabbit
Turtle 2 meets Rabbit
```

# Combining Multiple Flows

```
val hello = listOf("Hallo", "Hi")
val numbers = listOf("Fünf", "Five")
```

```
val helloFlow = hello.asFlow()
val numberFlow = numbers.asFlow()
```

Here a variety of *sequential* flow composition operators:

```
listOf(hello, numbers)
.flatten()
.forEach { println(it) }
```

```
Hallo
Hi
Fünf
Five
```

```
flowOf(helloFlow, numberFlow)
.flattenConcat()
.collect { println(it) }
```

```
hello.flatMap{ hi ->
    numbers.map{"$hi $it"}
}.forEach { println(it) }
```

```
Hallo Fünf
Hi Five
```

```
helloFlow.flatMapConcat { hi ->
    numberFlow.map { "$hi $it" }
}.collect { println(it)}
```

Many more sophisticated (non-sequential) flow combinator exist, like `zip`, `combine`, `merge`, `flattenMerge`, `flatMapMerge` and `flatMapLatest`

# Exercise: Flow


Open: Coroutines02FlowsExercise.kt and Coroutines02FlowsExerciseTest.kt

**Solve Exercise: A-D**

- `Coroutines02FlowsExerciseTest.kt`: here you find the instructions of all exercises, which most have to be completed right away in the test.
- `Coroutines02FlowsExercise.kt`: in this file you have to provide an implementation, which will be used in all exercises

You have completed the challenge if all tests succeed in `Coroutines02FlowsExerciseTest`.

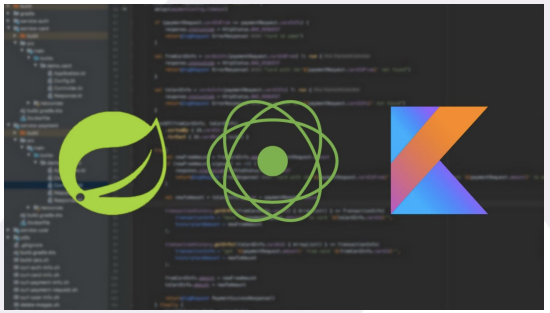
# Agenda

- Coroutines Basics: Structured Concurrency
- Coroutines Basics: Contexts + Propagation

- Coroutines & Spring Boot: Introduction
- Coroutines & Spring Boot: Reactive DB Access & REST calls with Coroutines
- Coroutines & Spring Boot: Reactive Controllers with Coroutines

- Coroutines: Flow
- Coroutines & Spring Boot: Reactive Stream Controllers with Flow

- Virtual Threads &| Coroutines



# Flow Support in Spring Boot

Reactor uses the following primitive to handle a *sequence of values* that *eventually become available over time*:

`Flux<T>`:

- `Flux<T>` can be considered a reactive counterpart of a collection

When using Coroutines:

- `Flux<T>` becomes `Flow<T>`

The `Flow` abstraction is considerably easier to manage than `Flux`. As you might know by now, `Flow` has a few methods you have to be aware of like `flow{...}`, `emit(...)` and `collect()`. Within a flow you can you use common language constructs to program its logic and do not have to rely on a complex API like in `Flux`.

spring BOOT

Kotlin Coroutines

# Database Access & Flow

Using `Flow<T>`, the contents of a large query - whose results would never fit in memory - can be returned.

```
@Repository
interface UserDao : CoroutineCrudRepository<User, Long> {

  fun findById_GreaterThan(id: Long) : Flow<User>

}
```

A method that returns a `Flow<T>` never has to be `suspend`.

Use the same common naming syntax or `@Query` annotations to define a query and return a `Flow<T>` instead of `Flux<T>` (aka `List<T>` in traditional repositories)

# Controllers & Flow

Simply return a `Flow<T>` (e.g. from a Dao/Service) and Spring boot knows how to handle it

```
@RestController
class DummyController(val dao:UserDao) {

    @GetMapping("/users/{from-id}")
    @ResponseBody
    fun userFlow(@PathVariable("from-id") id: Long): Flow<String> =
        dao.findById_GreaterThan(id)
}
```

Under the hood, `Flow<T>` will be transformed to a `Flux<T>`, similar like a `Mono<T>` is transformed to a `suspend` function.

# Controllers & Flow

```
@RestController
class DummyController(val dao:UserDao) {


    @GetMapping("/infinite")
    @ResponseBody
    fun infiniteFlow(): Flow<String>  = flow {
        generateSequence(0){it + 1}.forEach {
         emit("$it \n")
        }
    }
}
```

...or declare your own using the flow builder.

# Even fancier: Flow & ServerSentEvents

Turn your endpoint into a ServerSentEvent stream by using `Flow<T>` with `Content-Type: text/event-stream`

```
@RestController
class DummyController() {

    @GetMapping("/infinite/sse", produces = [MediaType.TEXT_EVENT_STREAM_VALUE])
    @ResponseBody
    fun sseFlow(): Flow<ServerSentEvent> = flow {
        generateSequence(0){it + 1}.forEach {
            emit(ServerSentEvent.builder<String>()
                .event("hello-sse-event")
                .id(it.toString())
                .data("Your lucky number is $it").build())
            delay(500L)
        }
    }
}
```

`id/event` is optional

`data` is mandatory


Hi! I'm **ServerSentEvents** - **SSE** for friends - and I'm already old, since I'm born in 2006 together with html 5!

# Even fancier: Flow & ServerSentEvents

Using standard `curl` you then get:

```
curl http://localhost:8081/infinite/sse

id:0
event:hello-sse-event
data:Your lucky number is 0

id:1
event:hello-sse-event
data:Your lucky number is 1

id:2
event:hello-sse-event
data:Your lucky number is 2

id:3
event:hello-sse-event
data:Your lucky number is 3

id:...
event:hello-sse-event
data:Your lucky number is …


data:
```

An empty data element denotes a *heartbeat*.

# Exercise: async / SSE Controllers & Coroutines

Open: StockTraderController and Coroutines03FlowControllerTest.kt

**Solve Exercise: A-C**

- `StockTraderController.kt`: find the instructions for exercises named Challenge 3 - Part 3 A-C.

- `Coroutines03FlowControllerTest.kt`: make all tests succeed.

You have completed the challenge if all tests succeed in `Coroutines03FlowControllerTest`.

Try out the endresult with the webfrontend:

- Start the application
- http://localhost:8081/stocks-management.html

# Exercise: Websockets & Coroutines

Open: StockTradingAdvisorController and Coroutines04WebsocketControllerTest.kt

**Solve Exercise: A-B**

- `StockTradingAdvisorController.kt`: find the instructions for exercises named Challenge 3 - Part 4 A-B.

- `Coroutines04WebsocketControllerTest.kt`: make all tests succeed.

You have completed the challenge if all tests succeed in `Coroutines04WebsocketControllerTest`.

Try out the endresult with the webfrontend:

- Set the `OPENAI_API_KEY` in the environment
- Start the application
- http://localhost:8081/stocks-advisor.html

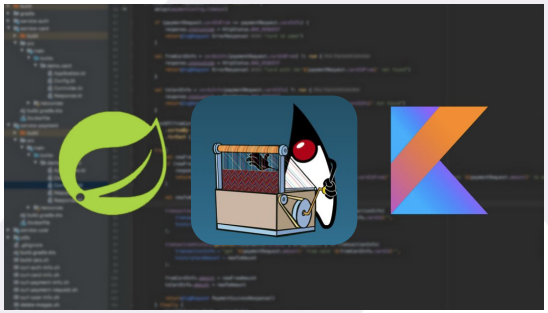
# Agenda

- Coroutines Basics: Structured Concurrency
- Coroutines Basics: Contexts + Propagation

- Coroutines & Spring Boot: Introduction
- Coroutines & Spring Boot: Reactive DB Access & REST calls with Coroutines
- Coroutines & Spring Boot: Reactive Controllers with Coroutines

- Coroutines: Flow
- Coroutines & Spring Boot: Reactive Stream Controllers with Flow

- Virtual Threads &| Coroutines



*Please help me refresh my memory:* ***could you do a quick recap of the benefits of Coroutines in Spring Boot****?*

# Spring Web: Sequential blocking programming approach

```
@GetMapping("/users")
@ResponseBody
@Transactional
fun storeUser(@RequestBody user:User): User {
    val avatarUrl  = randomAvatar()  //assuming a remote blocking call
    val validEmail = verifyEmail(user.email) //assuming a remote blocking call
    if(!validEmail) throw InvalidEmailException("Invalid email")
    return save(user.copy(avatar = avatarUrl)) //some blocking database call
}
```

# Spring Web: Sequential blocking programming approach

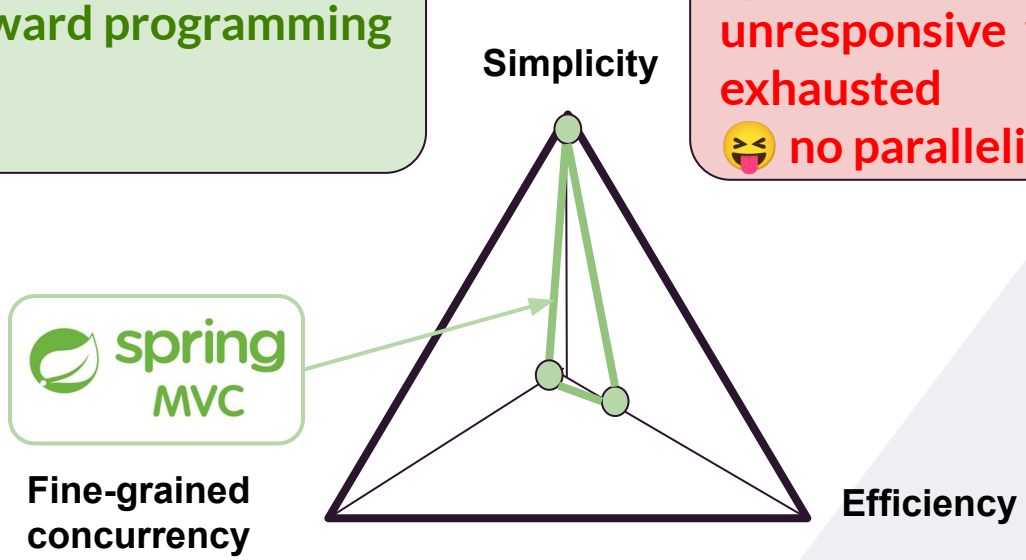
```
@GetMapping("/users")
@ResponseBody
@Transactional
fun storeUser(@RequestBody user:User): User {
    val avatarUrl  = randomAvatar()  //assuming a remote blocking call
    val validEmail = verifyEmail(user.email) //assuming a remote blocking call
    if(!validEmail) throw InvalidEmailException("Invalid email")
    return save(user.copy(avatar = avatarUrl)) //some blocking database call
}
```

spring MVC

😃 straightforward programming model

😝 inefficient: can get unresponsive when thread pool is exhausted

😝 no parallelism support



# Spring Webflux: Reactive Programming Approach


```
@PostMapping(value = ["/users"])
@ResponseBody
@Transactional
fun storeUser(@RequestBody user: User): Mono<User> {
    val avatarMono:Mono<AvatarDto> = randomAvatar() //remote async call
    val validEmailMono:Mono<Boolean> = verifyEmail(user.email) //remote async call
    return Mono.zip(avatarMono, validEmailMono)
            .flatMap { tuple: Tuple2<AvatarDto, Boolean> ->
              if (!tuple.t2)
                Mono.error(ResponseStatusException(BAD_REQUEST, "Invalid Email"))
              else save(user) //some async database call
            }
}
```

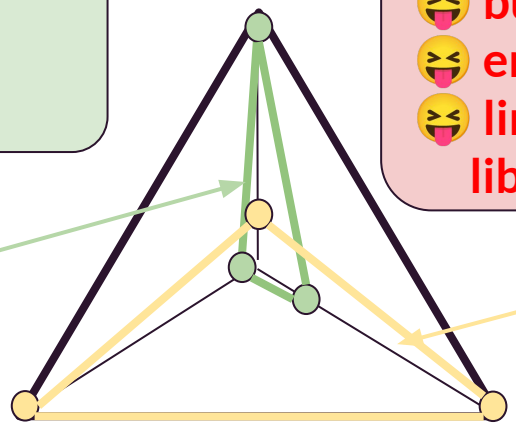
# Spring Webflux: Reactive Programming Approach

```
@PostMapping(value = ["/users"])
@ResponseBody
@Transactional
fun storeUser(@RequestBody user: User): Mono<User> {
    val avatarMono:Mono<AvatarDto> = randomAvatar() //remote async call
    val validEmailMono:Mono<Boolean> = verifyEmail(user.email) //remote async call
    return Mono.zip(avatarMono, validEmailMono)
            .flatMap { tuple: Tuple2<AvatarDto, Boolean> ->
              if (!tuple.t2)
                Mono.error(ResponseStatusException(BAD_REQUEST, "Invalid Email"))
              else save(user) //some async database call
            }
}
```

spring WebFlux

😀 resource efficient
😀 supports parallelism
😀 resilient

😝 complex programming model
😝 business intent gets lost
😝 error-prone abstractions
😝 limitation to non-blocking libraries (R2DBC)



# Spring Webflux with Coroutines: The best of both worlds


+


```
@RestController
class PersonController {

    @GetMapping("/persons")
    @ResponseBody
    @Transactional
    suspend fun storeUser(@RequestBody user:User): User = coroutineScope {
        val avatarUrl = async { avatarService.randomAvatar() }
        val validEmail = async { emailService.verifyEmail() }
        if(!validEmail.await()) throw InvalidEmailException("Invalid email")
        personRepo.save(user.copy(avatar = avatarUrl.await()))
}
```

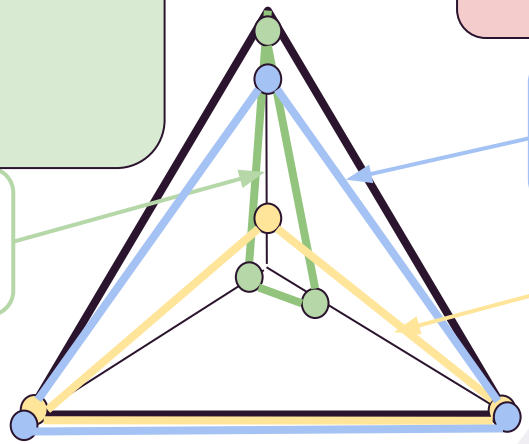
# Spring Webflux with Coroutines: The best of both worlds

```
@RestController
class PersonController {

    @GetMapping("/persons")
    @ResponseBody
    @Transactional
    suspend fun storeUser(@RequestBody user:User): User = coroutineScope {
        val avatarUrl = async { avatarService.randomAvatar() }
        val validEmail = async { emailService.verifyEmail() }
        if(!validEmail.await()) throw InvalidEmailException("Invalid email")
        personRepo.save(user.copy(avatar = avatarUrl.await()))
    }
```

spring WebFlux + Kotlin Coroutines

😀 straightforward programming model
😀 resilient
😀 resilient
😀 supports parallelism

😝 limitation to non-blocking libraries (R2DBC)




spring MVC

spring WebFlux + Kotlin Coroutines

spring WebFlux

# Coroutines Recap


- ☑ Lightweight Threads
- ☑ *Write synchronous logic* that is executed *asynchronously*
- ☒ Non-intrusive
- ☑ Structured Concurrency
  Automatic Synchronization, Context & Exception propagation across async processes
- ☑ Interop with Reactive/Async Frameworks

**All-in-all: Coroutines offer hassle-free concurrency**

*So far so good, but where do* ***Virtual Threads*** *fit in****?***

# Loom Factsheet



**Incubation**

- around 2018

**Motivation**

- Deliver JVM features and APIs for *easy-to-use*, *high-throughput lightweight concurrency* and new programming models to the Java platform

**Status**

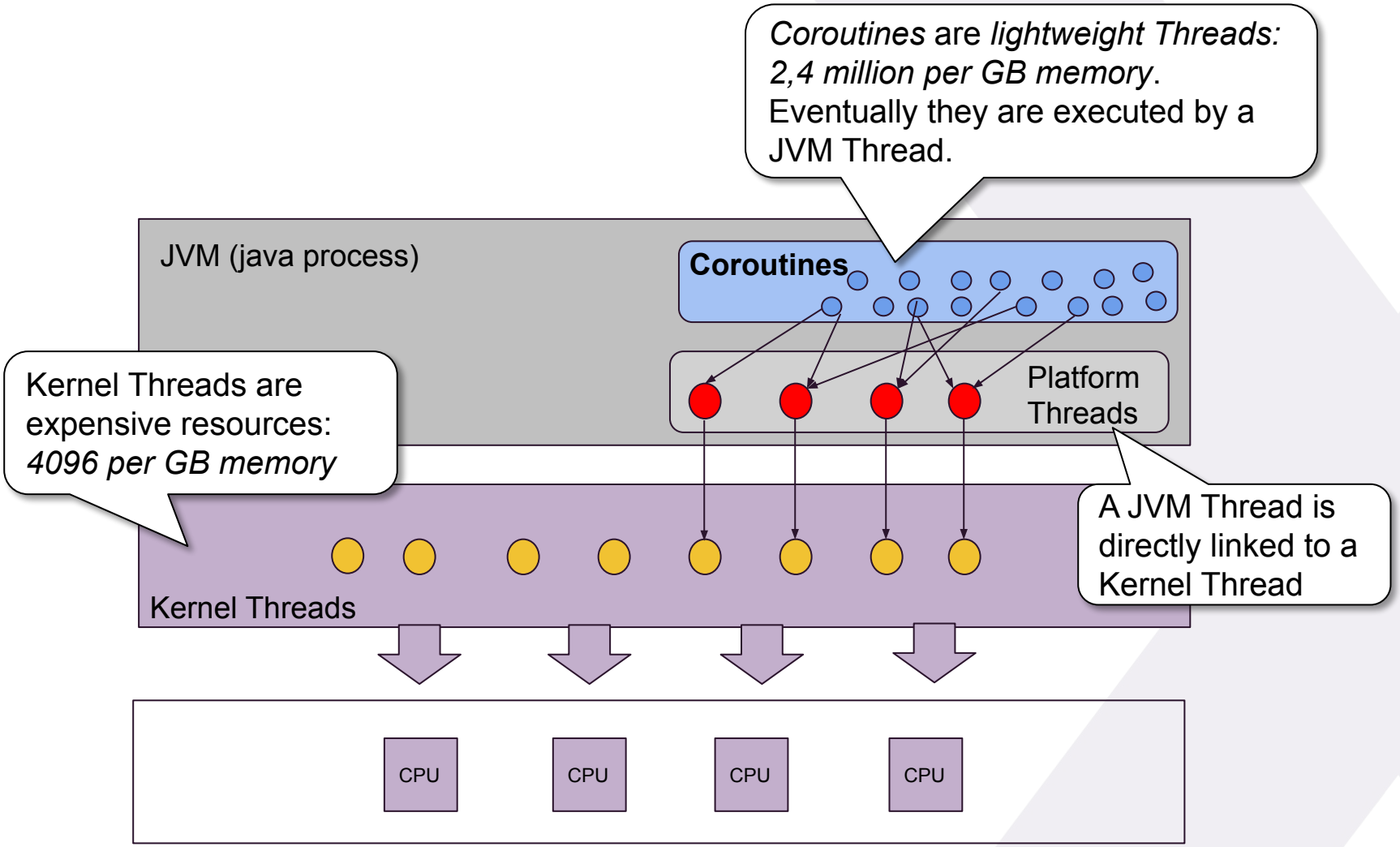
- JEP 425: **Virtual Threads** (Final) was integrated in JDK 19 - *final in JDK 21.*
  - similar to Coroutines
- JEP 428: **Structured Concurrency** (Incubator) was integrated in JDK 19.
  - similar to Kotlin's CoroutineScope
- JEP 429: **Scoped Values** (Incubator) has been integrated for JDK 20.
  - similar to Kotlin's CoroutineContext

# Ok, let's go explore Virtual Threads / Loom!

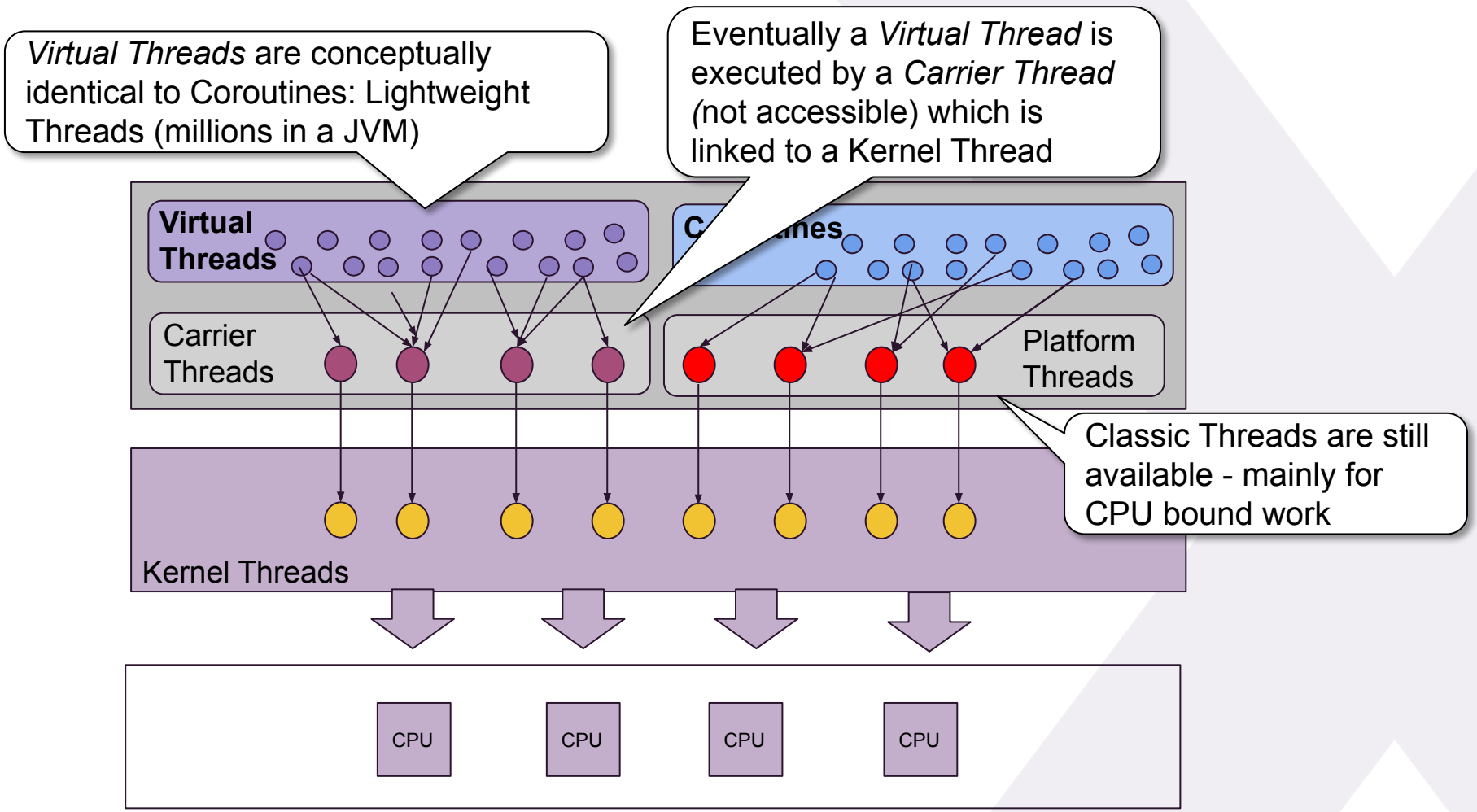
- ☐ Lightweight Threads
- ☐ *Write synchronous logic* that is executed *asynchronously*
- ☐ Non-intrusive
- ☐ Structured Concurrency
  Automatic Synchronization, Context & Exception propagation across async processes
- ☐ Interop with Reactive/Async Frameworks


**...and see which boxes can be ticked**

# Recap: What are Coroutines?



# Coroutines vs Loom's Virtual Threads - JEP 425 (JDK 21)



# How does a Virtual Thread look like?

```kotlin
val t1: Thread = thread {
    sleep(1000)
    println(Thread.currentThread().javaClass.simpleName)
}
```

Kotlin

Thread

```kotlin
val t2: Thread = Thread.startVirtualThread {
    sleep(1000)
    println(Thread.currentThread().javaClass.simpleName)
}
```

Kotlin

VirtualThread

Can you tell the difference?

# So what's the difference?

```
runBlocking {
    (1..100_000).map {
      launch {
         delay(2000)
         print(".")
       }
    }
    println("Coroutines: Ready to Roll")
}
```

```
val threads = (1..100_000).map {
   thread {
      sleep(2000)
      print(".")
   }
}
println("Threads: Ready to Roll")
threads.forEach{it.join()}
```


```
val threads = (1..100_000).map {
   Thread.startVirtualThread {
      sleep(2000)
      print(".")
   }
}
println("Virtual Threads: Ready to Roll")
threads.forEach{it.join()
```

`sleep()` behaves identical like `delay()`: it does not block a *kernel Thread*

# Virtual Threads Checklist

- [x] Lightweight Threads
- [ ] *Write synchronous logic* that is executed *asynchronously*
- [ ] Non-intrusive
- [ ] Structured Concurrency
  Automatic Synchronization, Context & Exception propagation across async processes
- [ ] Interop with Reactive/Async Frameworks

# We will - we will - Block you!

```
class AvatarService {

    fun randomAvatar(): Avatar =
        RestTemplate().getForEntity<Image>("http://<host>/avatar")
}
```

E.g. using the good old `RestTemplate` with a `VirtualThread` will perform like its reactive counterpart `WebClient`.

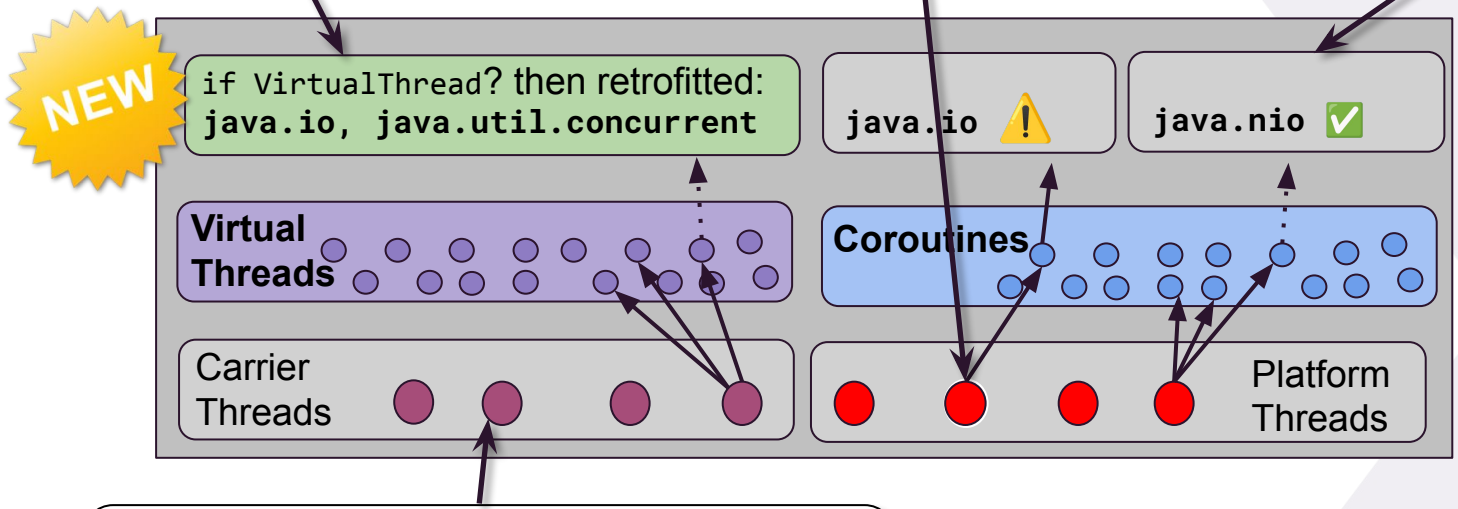
```
class AvatarService {
    fun randomAvatar(): Mono<Avatar> =
        WebClient.create("http://<host>")
                 .get()
                 .uri("/avatar")
                 .retrieve()
```

# Coroutines vs Loom's Virtual Threads

Whenever a `VirtualThread` uses `java.io` (and other blocking API's) it *won't be blocked*, but *suspended*, because their implementation has been *retrofitted* for `VirtualThread`

If a Coroutines/Reactive *must* use a `java.io` based API, then a *dedicated Thread* (e.g. `Dispatchers.IO`) must be used, so that *EventLoop Threads* are never blocked.

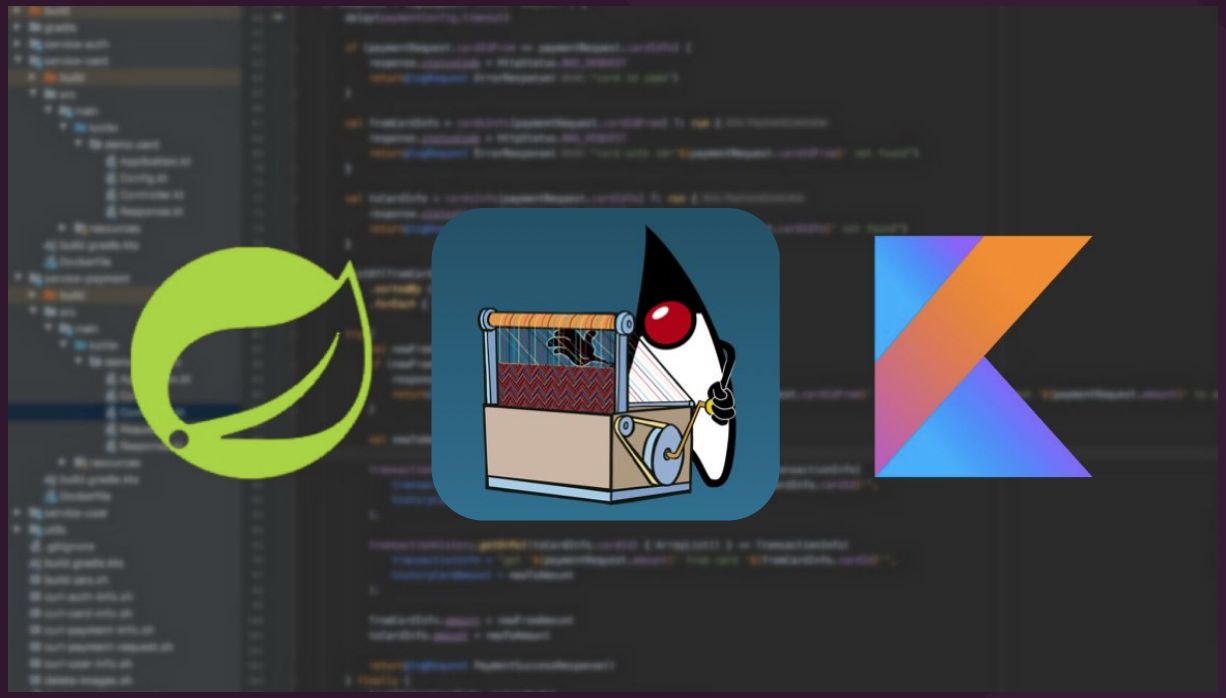
Coroutines/Reactive should not use `java.io` based API's like `RestTemplate` or `JDBC` but instead `java.nio` based API's like `WebClient`, `R2DBC` etc.




**Caution** ⚠️:
`synchronized {}` blocks and *native calls* will pin (= block) Carrier Threads!!!

# We will - we will - Block you!

# Spring Boot with Virtual Threads



Xebia

# Virtual Thread usage in Spring Boot Web

**Prerequisite**:

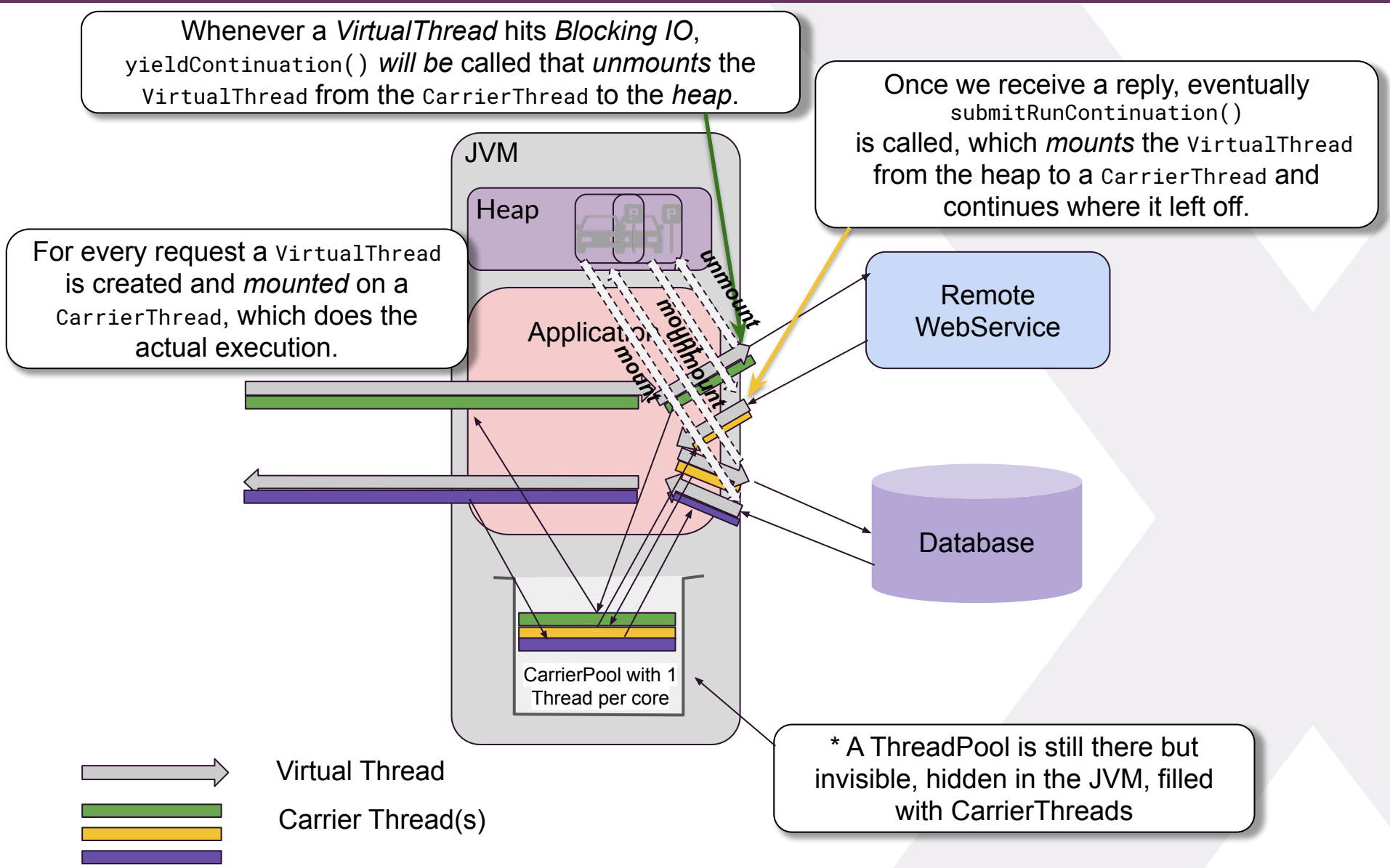
- Spring Boot 3.2+
- JDK 21+

**Configuration**:

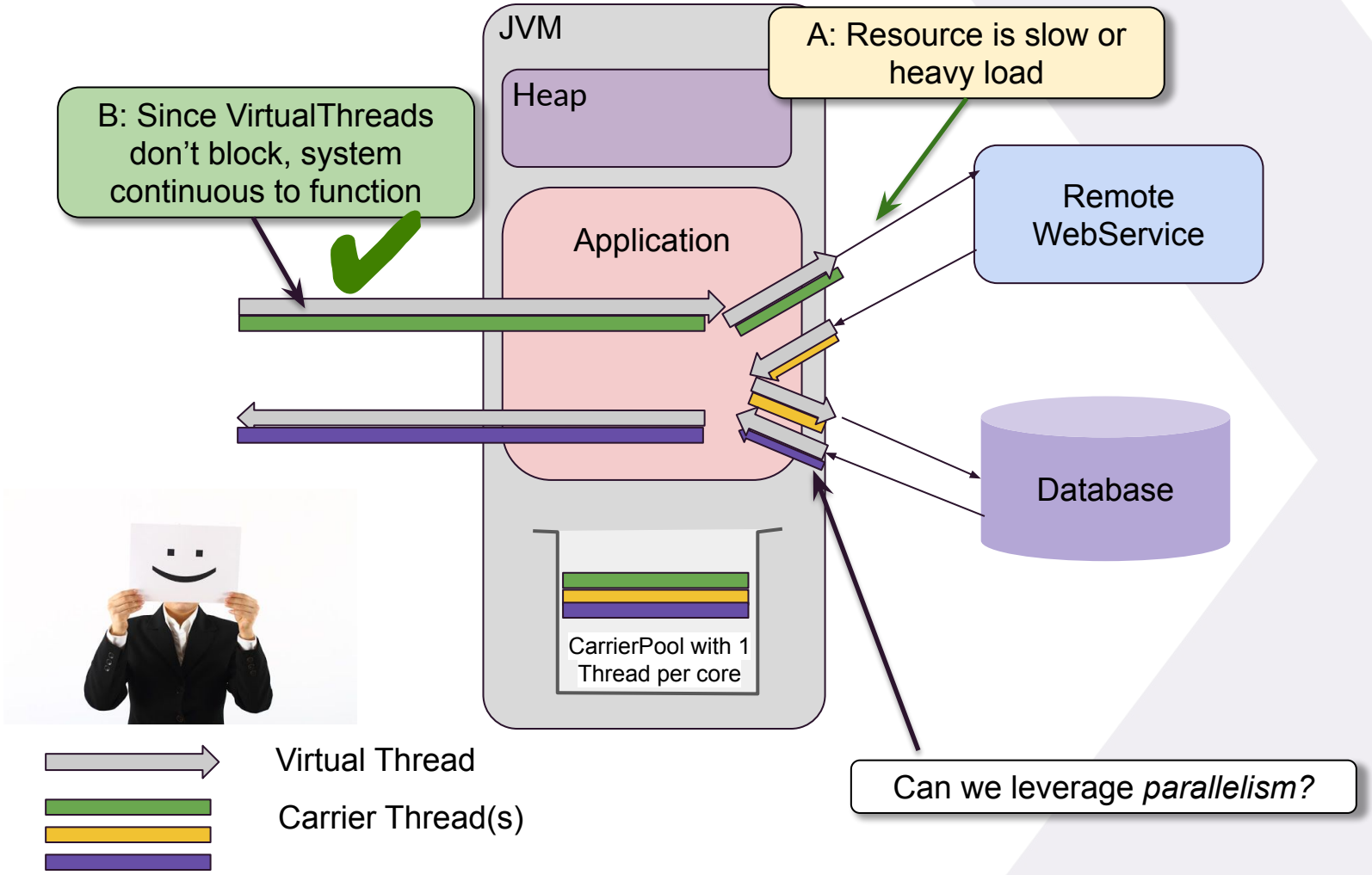
application.properties/.yaml

```
spring.threads.virtual.enabled=true
```


# Virtual Thread Application Architectures



# Virtual Thread Application Architectures



# Spring Web with Virtual Threads

```
@GetMapping("/users")
@ResponseBody
@Transactional
fun storeUser(@RequestBody user:User): User {
    val avatarUrl  = randomAvatar()  //blocking API call async with VT
    val validEmail = verifyEmail(user.email) //blocking API call async with VT
    if(!validEmail) throw InvalidEmailException("Invalid email")
    return save(user.copy(avatar = avatarUrl)) //blocking API call async with VT
}
```

# Spring Web with Virtual Threads

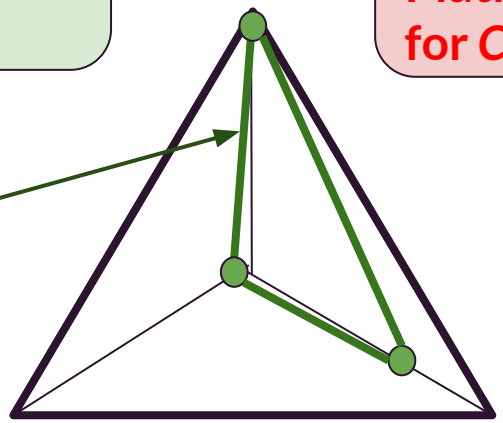
```
@GetMapping("/users")
@ResponseBody
@Transactional
fun storeUser(@RequestBody user:User): User {
    val avatarUrl  = randomAvatar()  //blocking API call async with VT
    val validEmail = verifyEmail(user.email) //blocking API call async with VT
    if(!validEmail) throw InvalidEmailException("Invalid email")
    return save(user.copy(avatar = avatarUrl)) //blocking API call async with VT
}
```

spring MVC

😀 Straightforward programming model
😀 resource efficient for *IO-bound work*

😝 no parallelism support
😝 harder to switch to Platform Threads that are required for *CPU-bound work*

spring MVC
with VirtualThreads



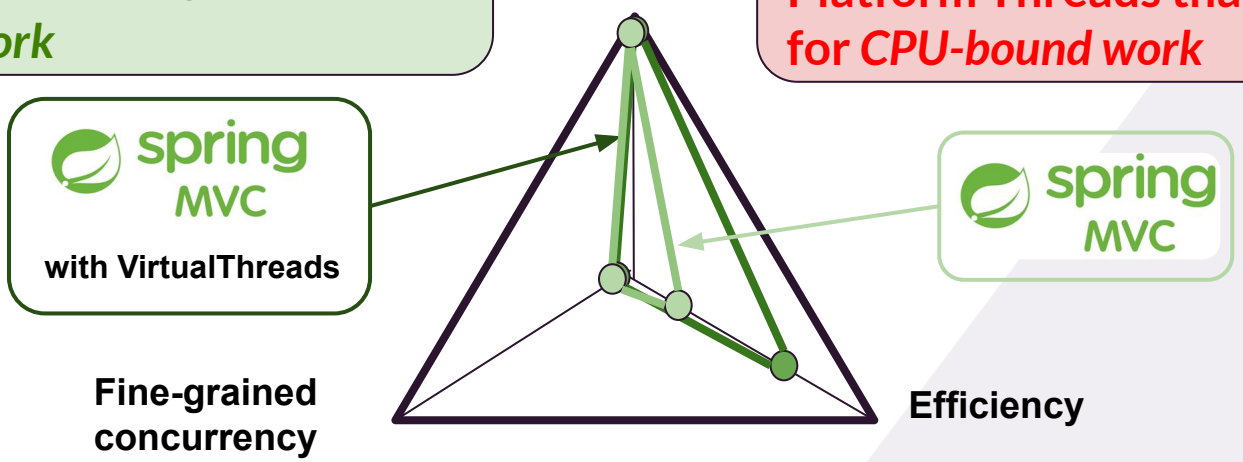
# Spring Web with Virtual Threads

```
@GetMapping("/users")
@ResponseBody
@Transactional
fun storeUser(@RequestBody user:User): User {
    val avatarUrl  = randomAvatar()  //blocking API call async with VT
    val validEmail = verifyEmail(user.email) //blocking API call async with VT
    if(!validEmail) throw InvalidEmailException("Invalid email")
    return save(user.copy(avatar = avatarUrl)) //blocking API call async with VT
}
```

spring MVC

😀 Straightforward programming model
😀 resource efficient for *IO-bound work*

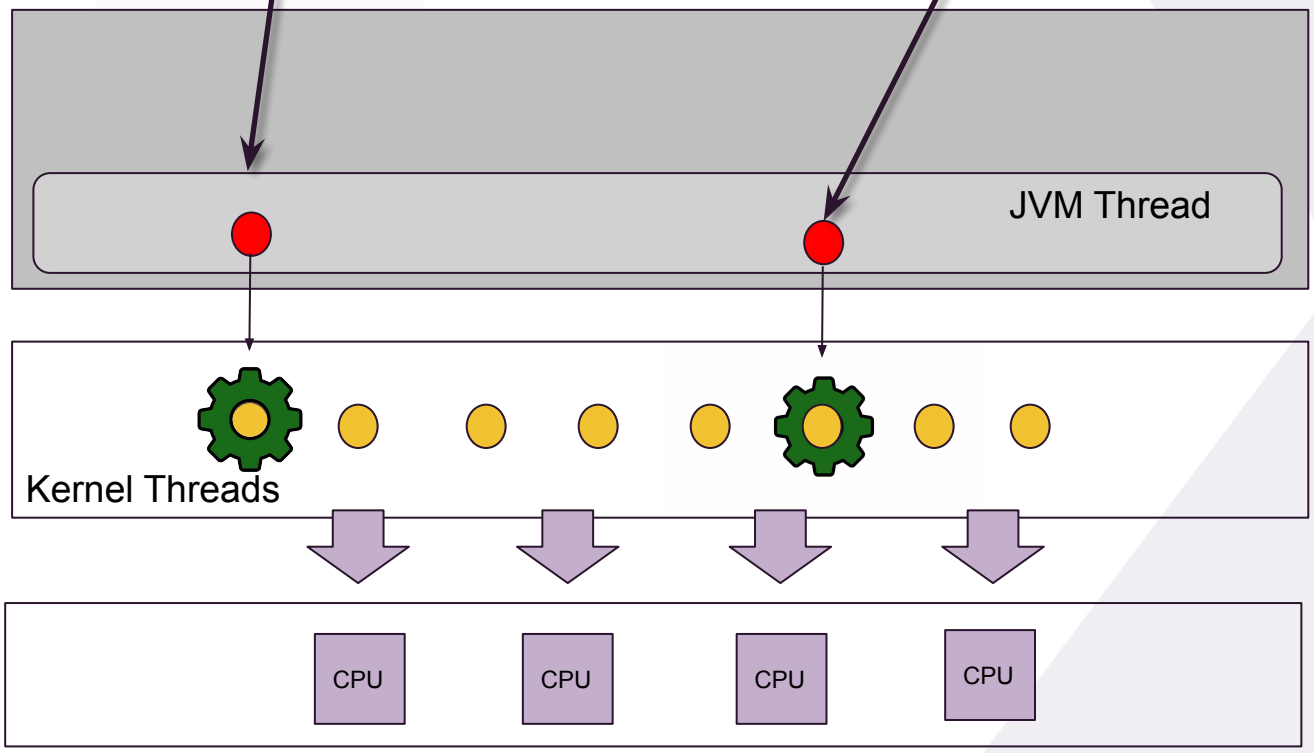
😝 no parallelism support
😝 harder to switch to Platform Threads that are required for *CPU-bound work*



# CPU-bound vs IO-bound work

**IO-Bound Work**
Task at hand depends on input/output operations.
Examples:
- Make a remote REST call
- Query a database
- Reading/Writing files
- Related: Waiting for a lock or sleep

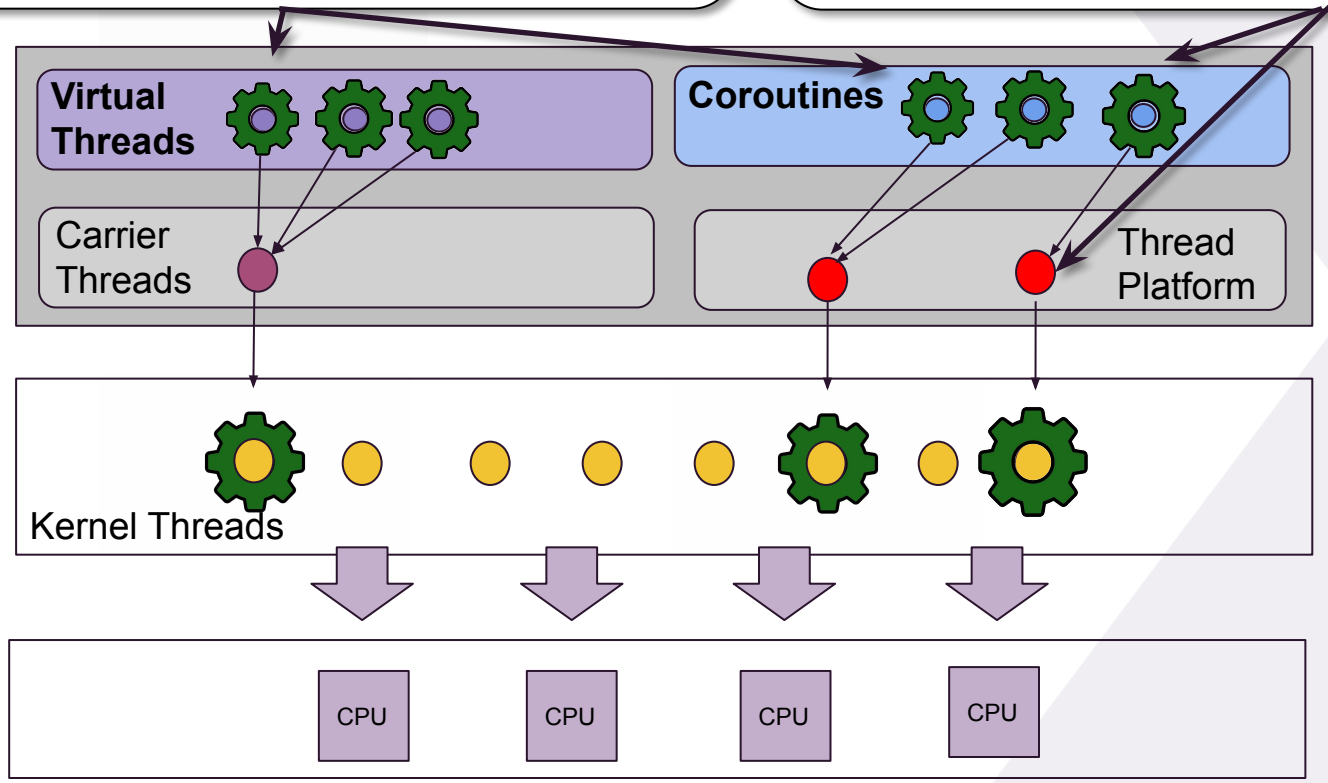
**CPU-Bound Work**
Task at hand mainly depends on CPU.
Examples:
- Mathematically calculations like:
- Cryptography
- Process complex algorithms
- train an AI model



# Virtual Threads are only a good match for IO-bound work

**IO-Bound Work**
Task at hand depends on input/output operations
Examples:
- Make a remote REST call
- Query a database
- Reading/Writing files
- Related: Waiting for a lock or sleep

**CPU-Bound Work**
Task at hand mainly depends on CPU.
Examples:
- Mathematically calculations like:
- Cryptography
- Process complex algorithms
- train an AI model



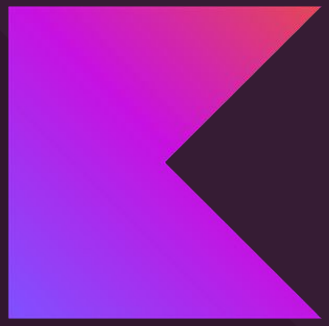
Perfect fit for
‘**Thread-Per-Request**’
type of applications

**Goals**

- Enable server applications written in the simple thread-per-request style to scale with near-optimal hardware utilization.
- Enable existing code that uses the `java.lang.Thread` API to adopt virtual threads with minimal change.

- ☑ Lightweight Threads
- ☑ *Write synchronous logic* that is executed *asynchronously*
- ☑ Non-intrusive
- ☐ Structured Concurrency
  Automatic Synchronization, Context & Exception propagation across async processes
- ☐ Interop with Reactive/Async Frameworks

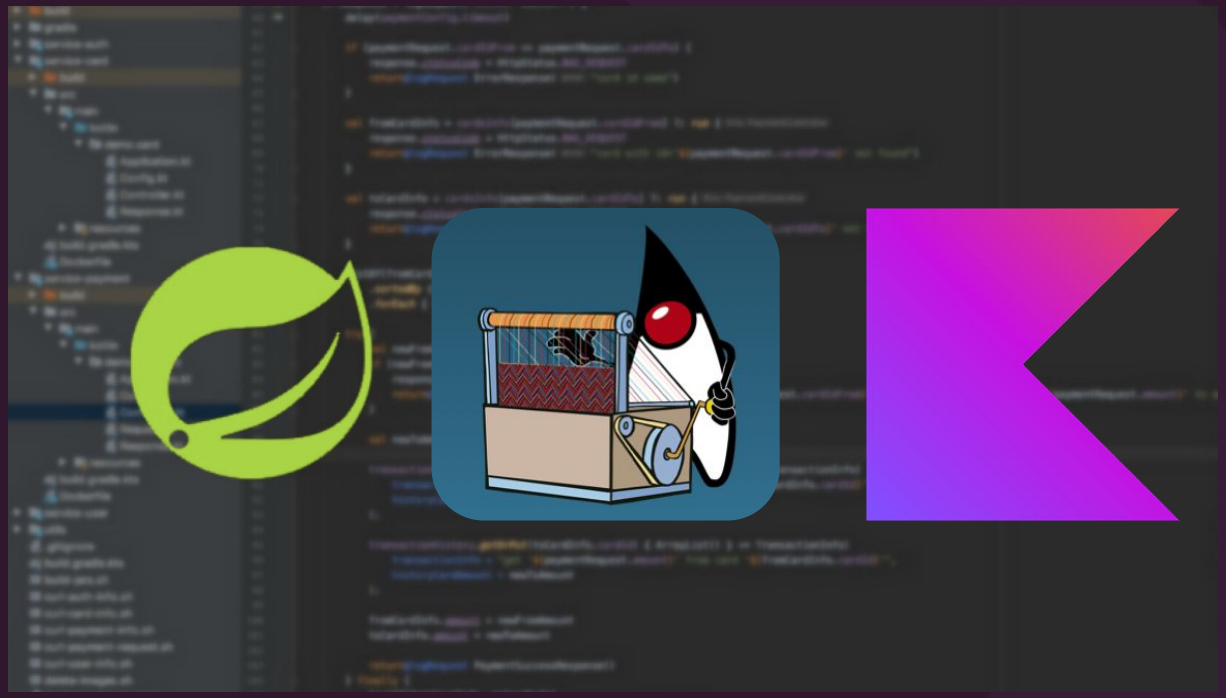
# Structured Concurrency with Loom



Forecast

*Last but not least: Can I combine Virtual Threads with Coroutines and / or reactive libraries?*

# Spring Boot with Coroutines combined with Virtual Threads



## The wrong way revisited

Xebia

# Coroutines & VirtualThreads the dangerous way in Spring Boot

```
@GetMapping("/users")
@ResponseBody
@Transactional
fun storeUser(@RequestBody user:User): User = runBlocking {
    val avatarUrl  = async { randomAvatarBlocking() } //async with VT 1
    val validEmail = async { verifyEmailBlocking() }//async with VT 1
    if(!validEmail.await()) throw InvalidEmailException("Invalid email")
    jpaRepository.save(user.copy(avatar = avatarUrl.await())) //sync with VT 1
}
```

spring MVC

I remember from the last time that the use of `runBlocking{...}` should be avoided at all costs

But with a `VirtualThread` nothings blocks, right?

*> What's the problem with the code above?*

# Coroutines & VirtualThreads the dangerous way in Spring Boot

```
@GetMapping("/users")
@ResponseBody
@Transactional
fun storeUser(@RequestBody user:User): User = runBlocking(Dispatchers.IO) {
    val avatarUrl  = async { randomAvatarBlocking() } //async with IO Thread 1
    val validEmail = async { verifyEmailBlocking() }//async with IO Thread 2
    if(!validEmail.await()) throw InvalidEmailException("Invalid email")
    jpaRepository.save(user.copy(avatar = avatarUrl.await())) //sync IO Thread 3
}
```

spring MVC

Ok, so let's use the `Dispatchers.IO` to ensure enough Threads are available to execute the Coroutines

*> What's the problem with the code above?*

# Coroutines & VirtualThreads the dangerous way in Spring Boot

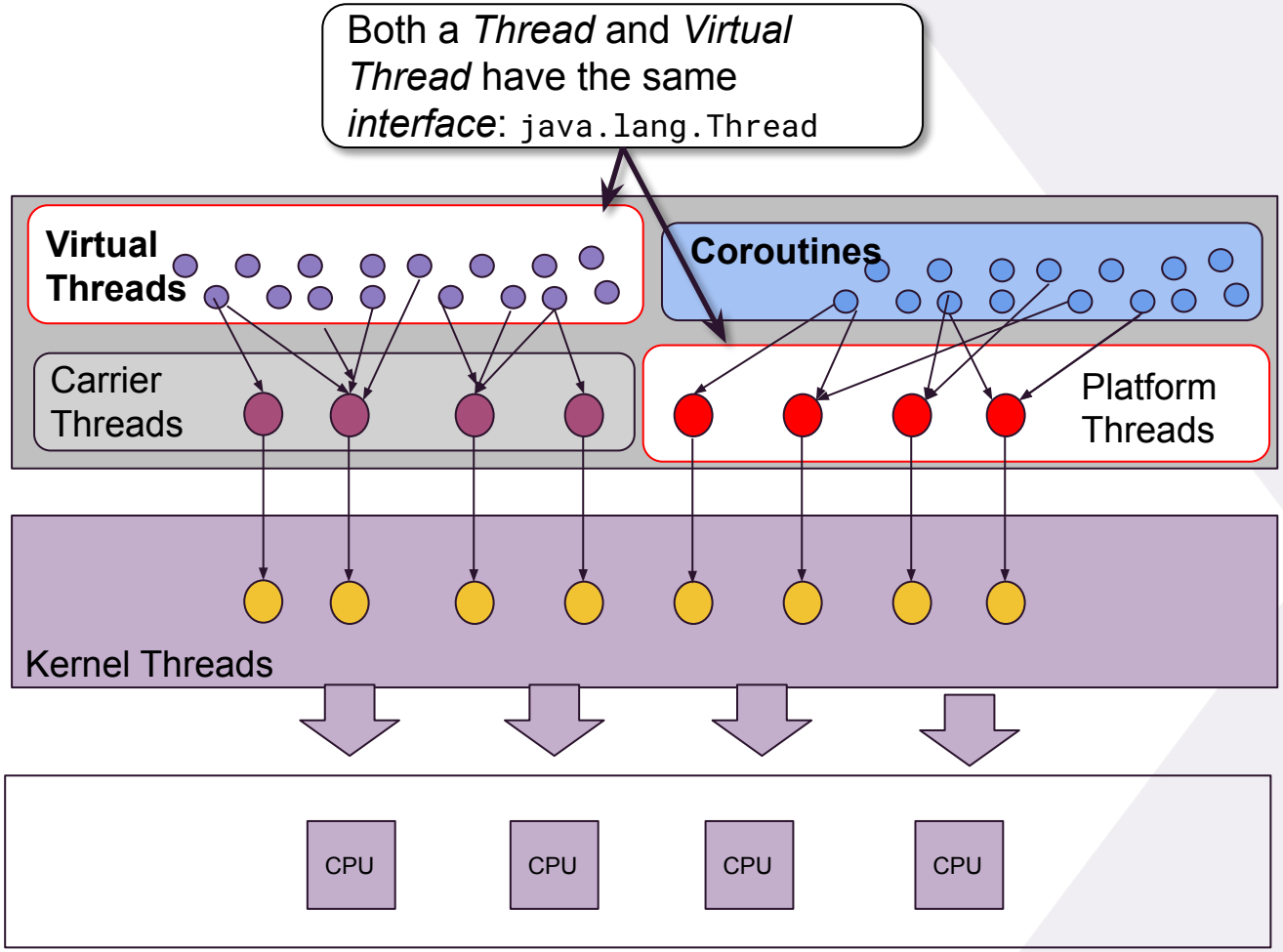
We can improve the situation by using a CoroutinesDispatcher that uses *Virtual Threads* to execute Cortoutines rather than a *Platform Threads*.

```
val Dispatchers.VT: CoroutineDispatcher
   get() = Executors.newVirtualThreadPerTaskExecutor().asCoroutineDispatcher()
```

This is how a VirtualThread Dispatcher is created.

spring MVC

# Virtual Threads from the Outside



# Coroutines & VirtualThreads the dangerous way in Spring Boot

```
@GetMapping("/users")
@ResponseBody
@Transactional
fun storeUser(@RequestBody user:User): User = runBlocking(Dispatchers.VT) {
    val avatarUrl  = async { randomAvatarBlocking() } //async with VT 1
    val validEmail = async { verifyEmailBlocking() }//async with VT 2
    if(!validEmail.await()) throw InvalidEmailException("Invalid email")
    jpaRepository.save(user.copy(avatar = avatarUrl.await())) //sync with VT 3
}
```

spring MVC

So finally: problem solved, right?

*> What's the problem with the code above?*

# Coroutines & VirtualThreads the dangerous way in Spring Boot

Nope, we lack context propagation!

Context propagation could be mitigated by providing (custom) `ThreadContextElement` implementations to propagate the above contexts, like the `MDCContext()`, `~TransactionContext()`, `~SecurityContext()` etc.

```
@GetMapping("/users")
@ResponseBody
@Transactional
fun storeUser(@RequestBody user:User): User = runBlocking(Dispatchers.VT + MDCContext() +
                                                          TransactionContext() + …) {
    val avatarUrl  = async { randomAvatarBlocking() } //async with VT 1
    val validEmail = async { verifyEmailBlocking() } //async with VT 2
    if(!validEmail) throw InvalidEmailException("Invalid email")
    save(user.copy(avatar = avatarUrl)) //async with VT 3
}
```

… however, this can be dangerous too:

e.g. doing parallel database operations using `async`/`await` causes Threading issues, because non-reactive Transactions are not Thread-Safe.

# Coroutines & VirtualThreads the dangerous way in Spring Boot

Using `runBlocking` with VirtualThreads in Spring Boot *Web* applications is possible but comes with *many limitations* and *one advantage*:

**Limitations**

- No automatic context propagation, must be done manually.
  - always include a `VirtualThread` Dispatcher

```
fun withMyContext(body: suspend CoroutineScope.() -> T): T =
    runBlocking(Dispatchers.VT + MDCContext() + …, body)
```

- For database transactions:
  - You must create your own `TransactionContext`
  - in `async{...}` blocks no database code must be called that relies on a transaction, since it causes Threading issues

No reactive Stream support (`Flow`)

**Advantage**

- You can use JPA/non-reactive Hibernate and other ThreadLocal based ORMs

**But, how does the preferred way look like?**

# Dangerous yet doable when ‘a bit of parallelism is required’

`async` with a `VirtualThread` dispatcher and corresponding `MDCContext()` and if needed `SecurityContext()` is possible for *non-database* operations

```
@GetMapping("/users")
@ResponseBody
@Transactional
fun storeUser(@RequestBody user:User): User = runBlocking {
    val avatarUrl  = async(Dispatchers.VT + MDCContext()) {
         randomAvatarBlocking() //async with VT 1
     }
    val validEmail = async(Dispatchers.VT + MDCContext()) {
         verifyEmailBlocking() //async with VT 2
     }
    if(!validEmail) throw InvalidEmailException("Invalid email")
    save(user.copy(avatar = avatarUrl)) //current servlet thread carrying all context
}
```

spring MVC

All database calls must be executed within the current thread since it carries the `TransactionContext()` (and all other contexts).

# Spring Boot with Coroutines with Virtual Threads

The right way

Xebia

# Coroutines with Virtual Threads the right way in Spring Boot

**Virtual Thread Options:**

1. **Use Virtual Threads to process incoming requests**
   - In Reactor this implies using Virtual Threads for the event-loop

application.properties/.yaml

```
spring.threads.virtual.enabled=true
```

2. **Use Virtual Threads for Blocking API calls**

```
val Dispatchers.VT: CoroutineDispatcher
    get() = Executors.newVirtualThreadPerTaskExecutor().asCoroutineDispatcher()

withContext(Dispatchers.VT) {
    myBlockingAPICall()
}
```

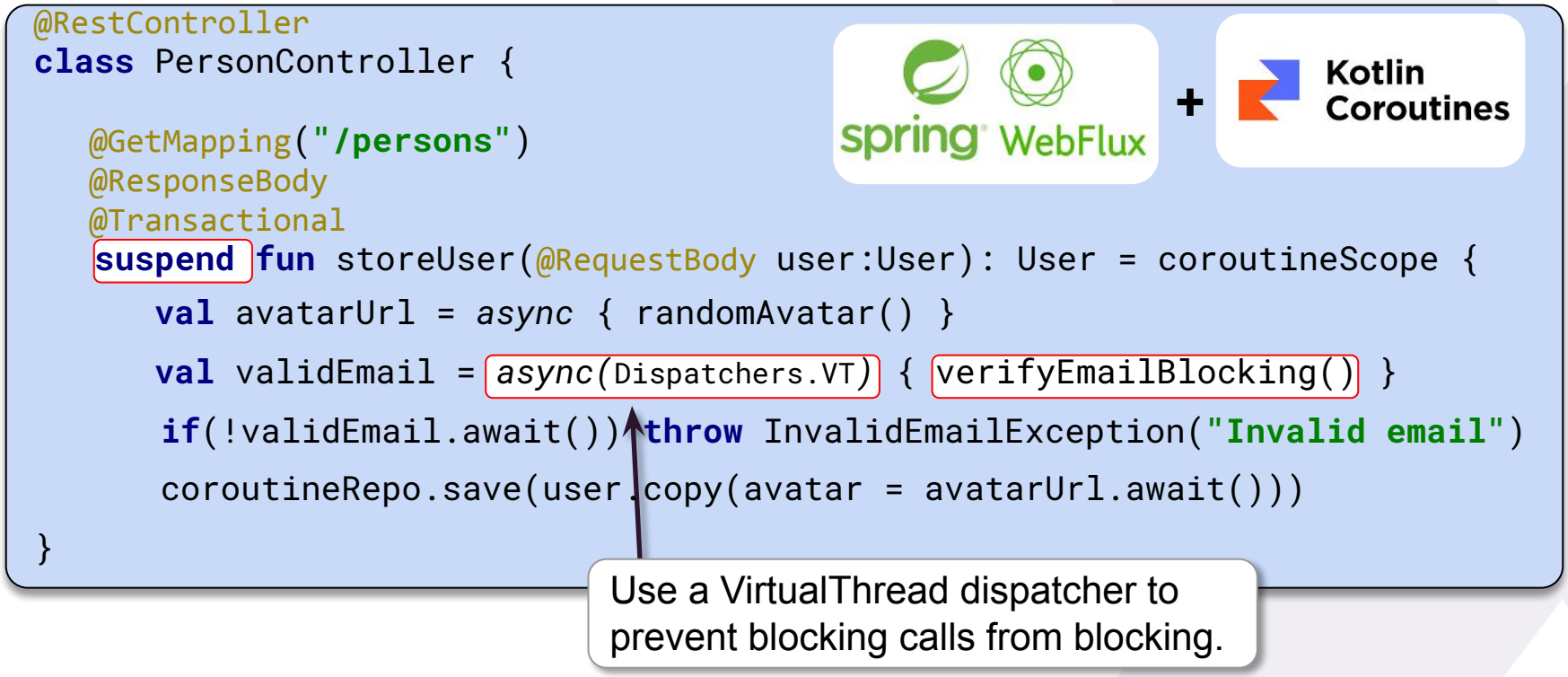
Use the VT `Dispatchers` to make blocking API calls

3. **…or both**

OK, but which one should I choose - if any…?

# Coroutines combined VirtualThreads in Spring Boot

VirtualThread Dispatchers can perfectly bridge unavoidable blocking calls in a reactive application.



# Exercise: Controllers & Coroutines

Open: ClassicStocksTraderController and CoroutinesVirtualThreads05ControllerTest.kt

**Solve Exercise: A-B**

- `ClassicStocksTraderController.kt`: find the instructions for exercises named Challenge 3 - Part 4 - Exercise A.

- `CoroutinesVirtualThreads05ControllerTest.kt`: Implement Exercise B and make all tests succeed.

You have completed the challenge if all tests succeed in `CoroutinesVirtualThreads04ControllerTest`.

# Forecast for Structured Concurrency

## Project Loom Status

- JEP 425: **Virtual Threads** (Final) was integrated in JDK 19 - *final in JDK 21.*
  - similar to Coroutines
- JEP 428: **Structured Concurrency** (Incubator) was integrated in JDK 19.
  - similar to Kotlin's CoroutineScope
- JEP 429: **Scoped Values** (Incubator) has been integrated for JDK 20.
  - similar to Kotlin's CoroutineContext

To try out the *incubator* features you must start the JVM with the following flags:

```
--enable-preview --add-modules jdk.incubator.concurrent
```

# Virtual Threads & Structured Concurrency (Incubator: JEP-428)

```
coroutineScope {
    launch { println("First: A") }
    launch { println("First: B") }
}
coroutineScope {
    launch { println("Second: A") }
    launch { println("Second: B") }
}
```

In Kotlin, Structured Concurrency is intuitively reflected by the structure of `coroutineScope{...}` blocks

```
try(var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    scope.fork(() -> {
        System.out.println("A: First");
        return null;
    } );
    scope.fork (() -> {
        System.out.println("A: Second");
        return null;
    } );
    scope.join();
    scope.throwIfFailed();
} catch (ExecutionException | 
    throw new RuntimeException(e);
}
try(var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    scope.fork(() -> {
        System.out.println("B: First");
        return null;
    } );
    scope.fork(() -> {
        System.out.println("B: Second");
        return null;
    } );
    scope.join();
    scope.throwIfFailed();
} catch (ExecutionException | InterruptedException e) {
    throw new RuntimeException(e);
}
```

Structured Concurrency is achieved with the `StructuredTaskScope` API

Java

# Virtual Threads & Structured Concurrency (Incubator: JEP-428)

```
coroutineScope {
  val r1 = async { "Turtle" }
  val r2 = async { "Rabbit" }
  println(
   "${r1.await()} meets ${r2.await()}")
}
```

```
try(var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    Future<String> r1 = scope.fork(() -> "Turtle");
    Future<String> r2 = scope.fork (() -> "Rabbit");
    scope.join();
    scope.throwIfFailed();
    System.out.println(r1.get() + "meets" + r2.get());
} catch (ExecutionException | InterruptedException e) {
    throw new RuntimeException(e);
}
```


To retrieve results of an asynchronous computation, Loom relies on the ‘blocking’ `Future` of 1.7, which in case of a `VirtualThread` is suspended rather than blocked.

# Structured Concurrency: Cancellation (Incubator: JEP-428)

```
coroutineScope {
  val r1 = async { "Turtle" }
  val r2 = async { "Rabbit" }
  r2.cancel()
  println("Sorry, cancelled")
}
```

```
try(var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    Future<String> r1 = scope.fork(() -> "Turtle");
    Future<String> r2 = scope.fork (() -> "Rabbit");
    r2.cancel(true)
    scope.join();
    scope.throwIfFailed();
    System.out.println("Sorry, cancelled");
} catch (ExecutionException | InterruptedException e) {
    throw new RuntimeException(e);
}
```


Calling the `Future`'s `cancel(...)` method will cancel all tasks in the given scope, which is *almost* identical to the Coroutines' cancel approach.

# Structured Concurrency: Exceptions & Cancellation (Incubator: JEP-428)

```
coroutineScope {
  launch {
     throw Exception("Bad Boy")
  }
  withContext(NonCancellable){
     launch {
       delay(1000)
       println("After 1s")
     }
  }
}
```

```
try(var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    scope.fork(() -> {
       throw new Exception("Bad Boy");
    });
    scope.fork (() -> {
       sleep(1000);
       System.out.println("No after 1s...");
       return null;
    });
    scope.join();
    scope.throwIfFailed();
} catch (ExecutionException | InterruptedException e) {
    throw new RuntimeException(e);
}
```

Java

⚠️ Loom does not allow tasks to be excluded from cancellation. This limitation will have consequences for many use-cases that require e.g. graceful shutdown.

- It is not a goal to replace the existing thread interruption mechanism with a new thread cancellation mechanism. We might propose to do so in the future.

  https://openjdk.org/jeps/428

# Structured Context Propagation (Incubator: JEP-429)

Loom does not offer an easy means to propagate `ThreadLocals` to other Threads like Coroutines do, but introduce a new concept called: `ScopeValue<T>`

```
MDC.put("request_id", "123")
logger.info("hallo root")
launch(MDCContext()) {
  logger.info("hallo sub")
}
```

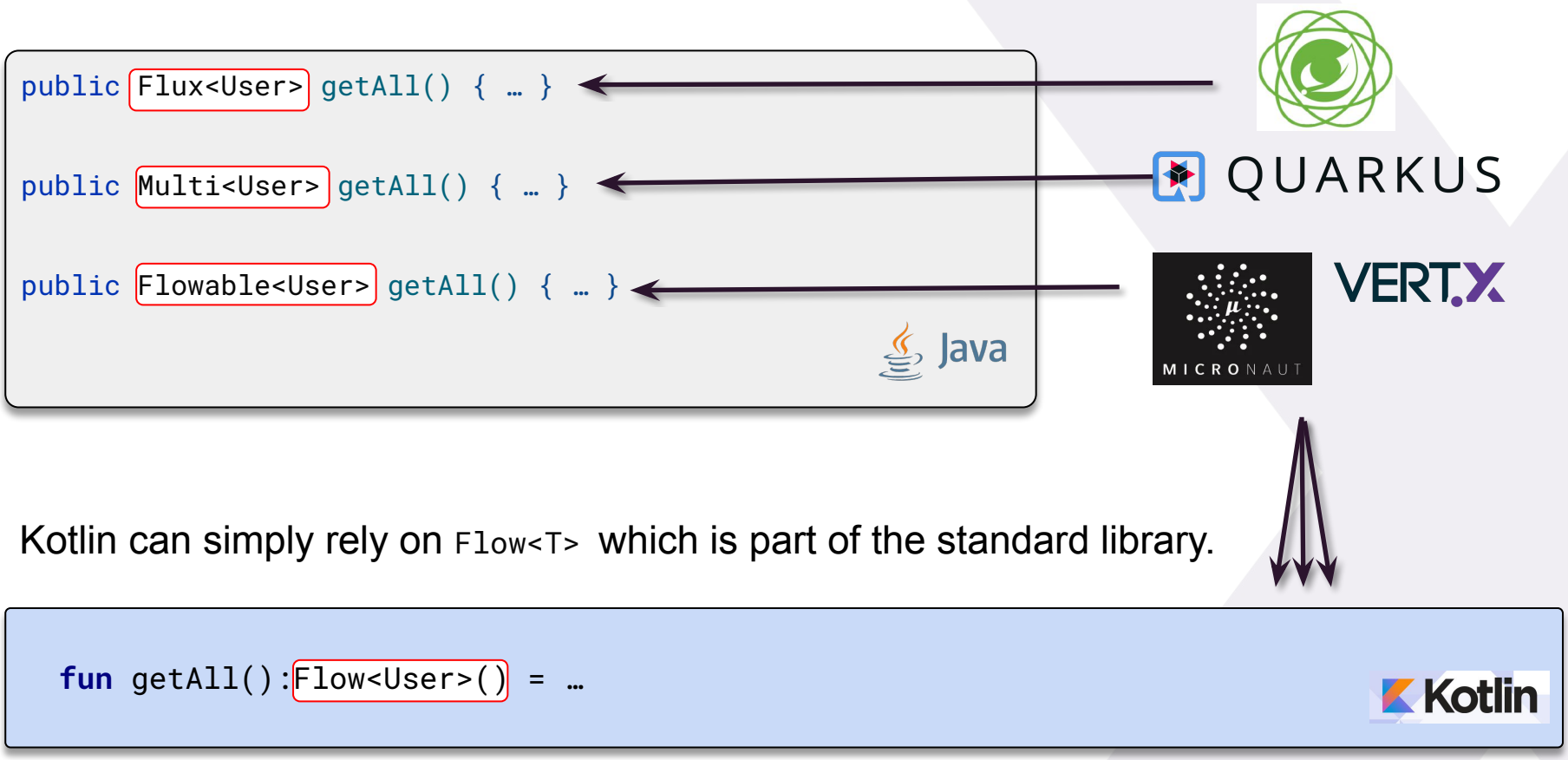
```java
ScopeValue<String> REQ_ID = ScopeValue.newInstance();
ScopeValue.where(REQ_ID, "123", () -> {
   LOG.info("hallo root " + REQ_ID.get());
   try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
       scope.fork(() -> {
           LOG.info("hallo sub" + REQ_ID.get());
           return null;
       });
       scope.join();
       scope.throwIfFailed();
   } catch (ExecutionException | InterruptedException e) {
       throw new RuntimeException(e);
   }
});
```

Java

⚠ Using a lambda to scope the area in which a certain state is accessible to multiple Threads can be *problematic* to retrofit existing `ThreadLocal` functionality in frameworks: namely in cases where different layers/components manipulate/use this state, e.g. web filter -> *sets state*, controller & services -> *gets state* etc.

# How Loom Structured Concurrency & Reactive Streams?

For *reactive streams,* Java still has to rely on one of the many reactive/async frameworks with their own abstractions out there:




Kotlin can simply rely on `Flow<T>` which is part of the standard library.

```
fun getAll():Flow<User>() = …
```

Kotlin

# Limitations of Structured Concurrency in Loom(Incubator: JEP-428 / JEP-429

⚠️ Loom's `StructuredTaskScope` does not support `ThreadLocal` *state propagation* across `VirtualThreads`:

- `ThreadLocal` state like `Transaction`, `SecurityContext`, `MDC` etc. won't work for parallel tasks 😕
- You can propagate `ThreadLocal` state manually, but it is tedious 😕
- `ScopeValue<T>` can't be retrofitted with current frameworks 😕
- Doing parallel operations won't work with `Transactions` since they are not Thread-Safe. 😕

⚠️ Task cancellation is limited:

- Excluding tasks from cancellation does not work 😕
- Cancellation is limited to `Thread` interruption 😕

# Verdict Loom's Structured Concurrency for now (Incubator: JEP-428 / JEP-429

✅ For simple *parallelism* (async/await) Loom's Structured Concurrency is much easier to learn than a reactive library

- Mostly Java devs profit from this
- For Kotlin devs it does not matter, since we have Coroutines already

❌ Loom's Structured Concurrency as is, is inferior to Kotlin Coroutines

- Cancellation, State Propagation, syntax

❌ It requires web-frameworks to be adjusted significantly

- ScopeValue vs ThreadLocal, multi-threaded Transactions

❌ Loom's Structured Concurrency has no primitive for reactive Streams

- You still have to use a reactive library / reactive web framework

# Virtual Threads Checklist

- [x] Lightweight Threads
- [x] *Write synchronous logic* that is executed *asynchronously*
- [x] Non-intrusive
- [~] Structured Concurrency
  Process Synchronization, Context & Exception propagation across async processes
- [ ] Interop with Reactive/Async Frameworks

# Copyright © 2026 Xebia BV All rights reserved